

Towards a Theoretically-Backed and Practical Framework for Selective Object-Sensitive Pointer Analysis

CHAOYUE ZHANG, Nanjing University, China
LONGLONG LU, Nanjing University, China
YIFEI LU*, Nanjing University, China
MINXUE PAN*, Nanjing University, China
XUANDONG LI, Nanjing University, China

Context sensitivity is a foundational technique in pointer analysis, critical and essential for improving precision but often incurring significant efficiency costs. Recent advances focus on selective context-sensitive analysis, where only a subset of program elements, such as methods or heap objects, are analyzed under context sensitivity while the rest are analyzed under context insensitivity, aiming to balance precision with efficiency. However, despite the proliferation of such approaches, existing methods are typically driven by specific code patterns, therefore lacking a comprehensive theoretical foundation for systematically identifying code scenarios that benefit from context sensitivity.

This paper presents a novel and foundational theory that establishes a sound over-approximation of the ground truth, i.e., objects that really improve precision under context sensitivity. The proposed theory reformulates the identification of this upper bound into graph reachability problems over a typical *Pointer Flow Graph* (PFG), each of which can be efficiently solved under context insensitivity, respectively. Building on this theoretical foundation, we introduce our selective context-sensitive analysis approach, Moon. Moon performs both backward and forward traversal on a *Variable Flow Graph* (VFG), an optimized variant of PFG designed to facilitate efficient traversal. This traversal systematically identifies all objects that improve precision under context sensitivity. Our theoretical foundation, along with carefully designed trade-offs within our approach, allows Moon to limit the scope of objects to be selected, leading to an effective balance between its analysis precision and efficiency. Extensive experiments with Moon across 30 Java programs demonstrate that Moon achieves 37.2X and 382.0X speedups for 2-object-sensitive and 3-object-sensitive analyses, respectively with negligible precision losses of only 0.1% and 0.2%. These results highlight that the balance between efficiency and precision achieved by Moon significantly outperforms all previous approaches.

CCS Concepts: • **Theory of computation → Program analysis**.

Additional Key Words and Phrases: Pointer Analysis, Selective Object-Sensitivity

ACM Reference Format:
Chaoyue Zhang, Longlong Lu, Yifei Lu, Minxue Pan, and Xuandong Li. 2025. Towards a Theoretically-Backed and Practical Framework for Selective Object-Sensitive Pointer Analysis. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 333 (October 2025), 28 pages. https://doi.org/10.1145/3763111

*Corresponding authors.

Authors' Contact Information: Chaoyue Zhang, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, zcy@smail.nju.edu.cn; Longlong Lu, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, dg20320003@smail.nju.edu.cn; Yifei Lu, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, lyf@nju.edu.cn; Minxue Pan, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, mxp@nju.edu.cn; Xuandong Li, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, lxd@nju.edu.cn.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2475-1421/2025/10-ART333
https://doi.org/10.1145/3763111

1 Introduction

Pointer analysis is a crucial technique in static analysis, providing the foundation for a wide range of applications, including bug detection [3, 28] and program understanding [22, 35]. Andersen-style pointer analysis is commonly recognized as the standard approach for context-insensitive analysis in Java [29, 34]. This analysis is performed on a typical pointer flow graph (PFG) [19, 21, 43, 44], where nodes represent pointers (variables and field pointers), and edges encode subset constraints between their points-to sets. While context-insensitive approaches are efficient, they often sacrifice precision. To address this, context-sensitive techniques have been introduced, which refine the analysis by considering calling contexts. However, these techniques can significantly increase the analysis overhead, especially when applied to large-scale programs, leading to potential scalability issues.

To address the challenges of context-sensitive pointer analysis, selective context sensitivity techniques have been introduced [7, 8, 19, 21, 24, 31]. These methods apply context sensitivity to specific elements, such as variables, methods, or heap objects, optimizing the trade-off between precision and efficiency. Recent research [7, 8] shows that applying context sensitivity selectively to heap objects yields particularly significant efficiency gains in sacrifice of little precision loss, often surpassing its application to other elements. However, current approaches remain largely empirical, relying on heuristics and ad hoc patterns to decide where context sensitivity should be applied. While effective, these methods lack a formal framework to explain why certain objects benefit more from context sensitivity than others, leading to ambiguity and inconsistency across programs. Moreover, heuristic-driven approaches, while useful in individual case studies, often fail to provide insights into the underlying principles governing object behavior under context sensitivity. As a result, these methods risk overemphasizing certain program components, leading to unnecessary computational overhead. A more comprehensive and theoretically grounded understanding of heap object behavior in the context of selective pointer analysis is necessary to refine these techniques and ensure their broader applicability.

In this paper, we establish a robust and accessible theoretical foundation that characterizes the conditions under which an object can improve precision in object-sensitive analysis. Fundamentally, our theory reformulates pointer analysis as a graph reachability problem on a *Pointer Flow Graph* (PFG) [19, 21, 25, 42, 44]. This reformulation enables us to define the *Ground Truth* for objects that genuinely contribute to precision improvement: an object is considered essential for precision if, upon removing only its contexts, some object(s) that previously could not reach a variable in the PFG becomes able to do so. Unfortunately, precisely identifying this ground truth is computationally intractable without conducting a real object-sensitive analysis due to inherent complexity. To address this, we introduce a two-step theory in this paper that soundly over-approximates the set of *ground truth* objects. The first step of our theory derives a provably sound over-approximation of the ground truth, termed *Precision-Relevant* objects. Our key insight is that ground truth objects are those whose field accesses result in partial, rather than complete, connectivity between the predecessors and successors in context-sensitive PFGs. Inspired by this insight, we define *Precision-Relevant* objects using key conditions based on reachability in PFGs. Among these, one condition ensures the partial connectivity, while the others refine the approximation by filtering out objects that lead to indistinguishable pointer flows, thereby eliminating redundancy. The second step of our theory extends this foundation to context-insensitive analysis for practical application, for which we propose the *Variable Flow Graph* (VFG), an optimized variant of PFG which models field operations and method calls with edges between associated variables, rather than the actual pointer flows, to facilitate efficient traversal. Based on VFGs, our theory provides a further upper-bound approximation of *Precision-Relevant* objects, whereas the reachability-based conditions on PFGs

are over-approximated by corresponding conditions on VFGs, ensuring computational feasibility without sacrificing theoretical rigor. To note, our theory is proven to be sound, ensuring that all objects contained in the ground truth can be included. Meanwhile, the approximations are carefully constructed to balance precision and scalability, making the theory both theoretically grounded and practically applicable.

Based on the theoretical foundation, this paper then proposes a practical framework for selective object-sensitive analysis, Moon. Specifically, Moon manages to check the reachability by backwards and forwards traversals on VFGs. Several tradeoffs have been made in the design of Moon, therefore enabling our Moon to identify most of *Precision-Relevant* objects with acceptable overhead. As a result, our approach Moon as well as the prototype tool with the same name is able to effectively pinpoint a small set of *Precision-Relevant* objects, eliminating the most of extra overhead while preserving nearly identical precision typically associated with conventional context-sensitive analysis.

Extensive experiments with Moon on four clients of pointer analysis across 30 Java programs demonstrate that our approach achieves 37.2X and 382.0X speedups for 2-object-sensitive and 3-object-sensitive analyses, respectively with negligible precision losses of only 0.1% and 0.2%. In comparison to other four SOTA related approaches, Moon exhibits comparable efficiency as the most efficient approach Cut-Shortcut [25], while achieving the precision level of the most precise approach DebloaterX [7]. Specifically, Moon achieves a significantly lower precision loss than Cut-Shortcut, which exhibits a precision loss of 62.8%, while achieving a 111.3% speedup over DebloaterX in 2-object-sensitive analysis. In summary, the contributions of this paper are as follows:

- A theoretical foundation on capturing all *Precision-Relevant* objects that may influence analysis precision in object-sensitive pointer analysis;
- A practical framework for object-sensitive pointer analysis, named Moon, that achieves balance between analysis precision and efficiency with carefully designed tradeoffs;
- A prototype implementation of Moon as well as all the source code and experimental data available as an artifact [46].
- A thorough evaluation of Moon on 30 Java programs, benchmarked against four SOTA approaches for comparative analysis.

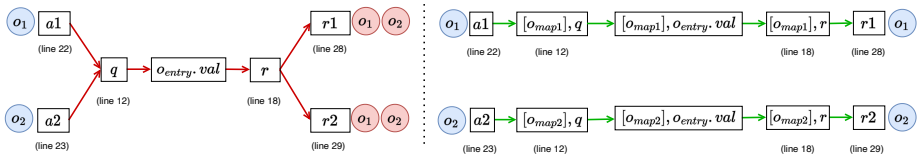
2 Motivating Example

Figure 1 presents a representative code snippet from the JDK, along with the corresponding Pointer Flow Graphs (PFG) under different kinds of context sensitivity. In a PFG, nodes represent objects and pointer variables, while edges denote points-to relationships. This example serves to: (1) demonstrate the fundamental role of heap objects in improving precision in object-sensitive analysis, (2) highlighting the limitations of state-of-the-art selective object-sensitivity approaches [7, 8], which misclassify certain redundant objects as requiring context sensitivity, leading to unnecessary analysis overhead, and (3) illustrate how our approach effectively differentiates objects that genuinely improve precision from those that do not.

The example features the **HashMap** class, which encapsulates an array of **Entry** objects as its field and provides **put()** and **get()** methods for storing key-value pairs and retrieving values, respectively. In the usage code, two objects o_1, o_2 are inserted into two separate **HashMap** instances o_{map1}, o_{map2} as both keys and values via **put()**, and later retrieved through **get()**. Ideally, after execution, result variables r_1, r_2 should point to their corresponding object o_1, o_2.

➤ **Q1: How does an object improve precision under object sensitivity?**

```
1  class Entry{
2    Object key, val;
3  }
4  class HashMap{
5    Entry[] table;
6    HashMap(){
7      this.table = new Entry[10];//o_table
8    }
9    void put(Object p, Object q){
10     Entry e = new Entry();//o_entry
11     e.key = p;
12     e.val = q;
13     Entry[] tbl = this.table;
14     tbl[0] = e;
15   }
16   Object get(Object k){
17     Entry en = this.table[0];
18     Object r = en.val;
19     return r;
20   }
21 void main(){
22   Object a1 = new Object();//o1
23   Object a2 = new Object();//o2
24   HashMap map1 = new HashMap();//o_map1
25   HashMap map2 = new HashMap();//o_map2
26   map1.put(a1, a1);
27   map2.put(a2, a2);
28   Object r1 = map1.get(a1);
29   Object r2 = map2.get(a2);
30 }
```



(*a*) Context Insensitivity (*b*) 2-Object-Sensitivity

Fig. 1. A motivating example from JDK

The PFGs in Figure 1 illustrate pointer flows through a field pointer $o_{entry}.val$ under different context sensitivities. Consider the PFG in sub-figure (b), where applying context sensitivity to heap object o_{entry} results in the replication of its instance field $o_{entry}.val$, leading to distinct replicas: $([o_{map1}], o_{entry}.val)$ and $([o_{map2}], o_{entry}.val)$. This ensures that pointer flows from different contexts remain isolated, preventing unintended merging, which is a problem inherent in context-insensitive analysis, as shown in sub-figure (a). Intuitively, heap objects improve precision by preserving the separation of distinct pointer flows through its field pointers under different contexts.

➤ Q2: What are the main limitations of state-of-the-art selective object-sensitivity approaches?

Existing selective object-sensitivity techniques rely primarily on identifying syntactic code patterns, failing to capture the deeper semantic reason why certain objects improve precision under context sensitivity while others do not. As a result, they overlook cases where multiple objects interact in a way that fails to separate pointer flows due to context length limitations. Consequently, part of these objects become redundant for precision improvement but still introduce significant overhead.

For example, suppose objects o_{map1} and o_{map2} have two distinct contexts, $[o_{c1}]$ and $[o_{c2}]$ (As o_{c1}, o_{c2} are hypothetical objects introduced for illustration, they are not shown in Figure 1). Now consider the pointer flow $q(\text{line } 12) \rightarrow o_{entry}.val \rightarrow r(\text{line } 18)$ and the true determinant for separating it is the field $o_{entry}.val$. Under 2-object-sensitivity, object contexts are limited to a length of 1, so the contexts of $o_{entry}.val$ will be $[o_{map1}]$ and $[o_{map2}]$, which will not include o_{c1} and o_{c2} by definition. As a result, the contexts of o_{map1} and o_{map2} (i.e., $[o_{c1}]$ and $[o_{c2}]$), become ineffective in separating this flow. Thus, context sensitivity applied to these map objects does not contribute to precision but merely incurs computational overhead.

➤ Q3: How does our approach achieve fine-grained object selection?

To address these issues, an effective object selection strategy for selective object-sensitivity must account for (1) the fundamental mechanisms by which objects improve precision and (2) the

methods	$m \in \mathbb{M}$	contexts	$c = [o, o'] \in \mathbb{C} = \bigcup_{i=0}^{k} \mathbb{O}^i$
instructions labels	$i, l \in \mathbb{L}$		
variables	$x, y \in \mathbb{V}$	contextual heap objects	$(c, o), (c', h) \in \mathbb{C} \times \mathbb{O}$
heap objects	$o, h \in \mathbb{O}$	contextual pointers	$(c, x), (c', h.f) \in \mathbb{C} \times \mathbb{P}$
fields	$f \in \mathbb{F}$	points-to relations	$pt : \mathbb{P} \to \mathcal{P}(\mathbb{O})$
pointers	$x, h.f \in \mathbb{P} = \mathbb{V} \cup (\mathbb{O} \times \mathbb{F})$	contextual points-to relations	$pt : \mathbb{C} \times \mathbb{P} \to \mathcal{P}(\mathbb{O})$
context length	$k \in \mathbb{N}$	fully contextual points-to relations	$\hat{pt} : \mathbb{C} \times \mathbb{P} \to \mathcal{P}(\mathbb{C} \times \mathbb{O})$

Fig. 2. Domains and notations of Object-Sensitive pointer analysis.

$$\frac{i : x = \text{new } T \text{ in } m \quad c \in \text{CTXS}(m)}{(\lceil c \rceil_{k-1}, o_i) \to (c, x)} \textbf{[New]} \qquad \frac{x = y \text{ in } m \quad c \in \text{CTXS}(m)}{(c, y) \to (c, x)} \textbf{[Assign]}$$

$$\frac{x.f = y \text{ in } m \quad c \in \text{CTXS}(m) \quad (c', h) \rightsquigarrow (c, x)}{(c, y) \to (c', h.f)} \textbf{[Field-Store]}$$

$$\frac{y = x.f \text{ in } m \quad c \in \text{CTXS}(m) \quad (c', h) \rightsquigarrow (c, x)}{(c', h.f) \to (c, y)} \textbf{[Field-Load]}$$

$$\frac{\begin{array}{c} l : r = x.k(a) \text{ in } m \quad c \in \text{CTXS}(m) \quad (c', o) \rightsquigarrow (c, x) \\ m' = \text{DISPATCH}(o, l), c^{m'} = \text{CONS}(c', o) \end{array}}{(c', o) \to (c^{m'}, this_{m'}) \quad (c, a) \to (c^{m'}, p_{m'}) \quad (c^{m'}, r_{m'}) \to (c, r)} \textbf{[Method-Call]}$$

Fig. 3. Rules of Construction of Pointer Flow Graph under Object-Sensitivity

interplay between object contexts and length limits when separating pointer flows. Unlike prior pattern-based approaches, which provide only a partial and syntactic view, our method derives a principled selection strategy by leveraging a theoretical framework that systematically identifies a modest yet effective subset of objects. Moreover, by modeling the inter-dependencies among object contexts and incorporating context length limits, our approach eliminates redundant objects, enabling a fine-grained selection process. This significantly accelerates selective object-sensitive pointer analysis while introducing little precision loss.

3 Problem Formulation

Pointer analysis models all pointers and objects in a program and propagates objects through pointers. Figure 2 defines the domains and notations of Object-Sensitive pointer analysis. Based on these definitions, we formalize pointer analysis as an equivalent reachability problem on *Pointer Flow Graph* (PFG) [19, 21, 25], the latest advancement in pointer analysis.

3.1 Pointer Analysis on Pointer Flow Graph

Definition 3.1. ***(Pointer Flow Graph (PFG))**. A pointer flow graph is a directed graph $G = (N, E)$, where $N \subseteq \mathbb{C} \times (\mathbb{P} \cup \mathbb{O}), E \subseteq N \times N$.*

Definition 3.2. ***(Reachability on PFG)**. We define several key kinds of reachabilities on PFG. Given a PFG G, $s \in \mathbb{O} \cup \mathbb{P}, c_s, c_t \in \mathbb{C}$ and $t \in \mathbb{P}$, if (c_t, t) is reachable from (c_s, s), it is denoted as $(c_s, s) \rightsquigarrow_G (c_t, t)$; otherwise, it is denoted as $(c_s, s) \not\rightsquigarrow_G (c_t, t)$. The reachability predicate can be generalized as follows:*

$$s \rightsquigarrow_G t := \exists c_s, c_t \in \mathbb{C} : (c_s, s) \rightsquigarrow_G (c_t, t) \quad s \not\rightsquigarrow_G t := \nexists c_s, c_t \in \mathbb{C} : (c_s, s) \rightsquigarrow_G (c_t, t)$$

Specifically, we replace the symbol $\rightsquigarrow$ with $\rightarrow$ if the two nodes s and t are adjacent in G. If there is no ambiguity about which PFG we discuss reachability in, $\rightsquigarrow_G$ can be written as $\rightsquigarrow$.

Definition 3.1 formally introduces PFG, wherein nodes correspond to objects and pointers, while edges represent the points-to relationships among these nodes. Building upon this definition, object-sensitive pointer analysis can be formalized as a reachability problem on a PFG. Specifically, whether a given contextual pointer $(c_1, x_1) \in \mathbb{C} \times \mathbb{P}$ may point to another contextual object/pointer $(c_2, x_2) \in (\mathbb{C} \times \mathbb{O}) \cup (\mathbb{C} \times \mathbb{P})$ is equivalent to determining whether the corresponding node of (c_1, x_1) in the PFG is reachable from the other corresponding node of (c_2, x_2), as formalized in Definition 3.2.

Rules guiding the construction of a PFG under object-sensitivity are illustrated in Figure 3, where we define three auxiliary functions: (1) CTXS: $\mathbb{M} \rightarrow \mathcal{P}(\mathbb{C})$, which returns the contexts of a given method; (2) DISPATCH: $\mathbb{O} \times \mathbb{L} \rightarrow \mathbb{M}$ resolves an instance method call to a target method; and (3) CONS: $\mathbb{C} \times \mathbb{O} \rightarrow \mathbb{C}$ derives new contexts for the callee method based on the receiver object and its current context. Specifically, given a context $c = [o_1, \ldots, o_n]$ and an object o, we define $\text{CONS}(c, o) = \lceil c :: o \rceil_k$, where $c :: o$ denotes the sequence obtained by appending o to the end of context c, and $\lceil \cdot \rceil_k$ denotes truncation to retain only the last k elements of the sequence. Similarly, in rule **[New]**, $\lceil c \rceil_{k-1}$ denotes truncation to retain only the last k-1 elements of context c. Both truncation operations are defined in consistent with a k-object-sensitive pointer analysis [30]. Informally, k-object-sensitivity models calling contexts by tracking up to k object allocation sites. As k increases, the analysis achieves higher precision since longer allocation site sequence is used as contexts to distinguish the objects/pointers. Notably, both h and o are used to denote heap objects in the program, whereas h is used to emphasize its role in propagating other objects through its field pointers, whereas o highlights its role as an object being propagated. To note, our formalism aligns with the ones appearing in existing literature [27, 29, 34]. Particularly, context-insensitive analysis can be viewed as a special case where all the contexts of nodes in PFG are empty, i.e., $N \subseteq \{\emptyset\} \times (\mathbb{P} \cup \mathbb{O})$.

For simplicity, we exclude pointers that do not point to any objects in a PFG, since these pointers are incapable of establishing aliases with other pointers, and therefore introduce no constraints or ambiguities to the points-to relations.

3.2 Selective Object-Sensitive Pointer Analysis

The rules of Selective Object-Sensitive Pointer Analysis are basically the same with those in Figure 3, except for the **[New]** rule, which is replaced by the $\textbf{[New]}_{sel}$ rule defined as below. The revised rule $\textbf{[New]}_{sel}$ takes as input a set $\mathbb{H}_{pr}$ of heap objects. Only objects in $\mathbb{H}_{pr}$ retain their contexts, while the contexts of all the other objects are discarded.

$$\frac{i : x = \text{new } T \text{ in } m \quad c \in \text{CTXS}(m) \quad c' = \lceil c \rceil_{k-1} \text{ if } o_i \in \mathbb{H}_{pr} \text{ else } [\,]}{(c', o_i) \rightarrow (c, x)} \textbf{[New]}_{sel}$$

DEFINITION 3.3. ***(PFG of selective object-sensitive pointer analysis).*** *We denote the PFG of a selective object-sensitive pointer analysis as $G_{\mathbb{H}}$, where $\mathbb{H} \subseteq \mathbb{O}$ denotes the set of objects applied with context sensitivity. Specially, we denote the PFG of conventional object-sensitive analysis using all objects as $G_{\mathbb{O}}$.*

3.3 Ground Truth of Selective Object-Sensitive Analysis

The ultimate goal of selective object-sensitive analysis is to maximize precision while minimizing the number of objects applied with context sensitivity, i.e., identifying only those that really improve precision. Since pointer analysis has been formalized as a graph reachability problem on PFGs

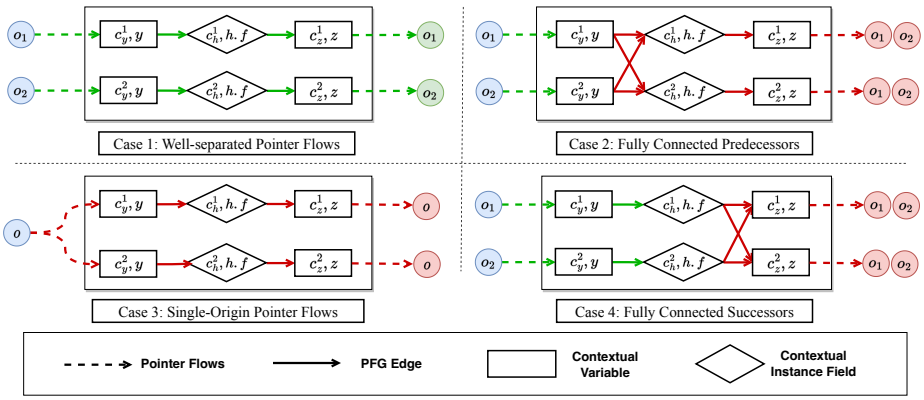


Fig. 4. The insights of whether an object can improve precision under context sensitivity

(as described in Section 3.1), the precision of the analysis can be characterized by the reachability between heap objects and variables. Building on this foundation, we formally define the *ground truth of selective object-sensitive analysis* as $\{h \in \mathbb{O} \mid \text{Precision} \uparrow (h)\}$. This ground truth denotes an ideal set of heap objects whose context-sensitive treatment contributes to precision improvement. That is to say, including any other objects outside this set does not lead to further precision gains. The predicate Precision $\uparrow$ is defined as below.

Definition 3.4. Precision $\uparrow$ *is a predicate on a heap object* h *to assert whether it change the reachability of an object between the conventional object-sensitive PFG* $G_{\mathbb{O}}$ *and the PFG* $G_{\mathbb{O}\setminus\{h\}}$ *that excludes object* h*.*

$$\text{Precision} \uparrow (h) := \exists v \in \mathbb{V}, o \in \mathbb{O} : o \not\rightsquigarrow_{G_{\mathbb{O}}} v \ \wedge o \rightsquigarrow_{G_{\mathbb{O}\setminus\{h\}}} v$$

As defined in Definition 3.4, an object should be applied to be context-sensitive if its exclusion from context-sensitive treatment introduces spurious points-to relationships that do not exist under a conventional object-sensitive analysis. Unfortunately, direct computation of reachability of all objects and pointers is infeasible, as neither $G_{\mathbb{O}}$ nor $G_{\mathbb{O}\setminus\{h\}}$ can be explicitly constructed without performing a real object-sensitive analysis. Alternative approaches must instead approximate without fully materializing the underlying graphs.

4 Upper-Bound Approximation for Ground Truth of Selective Object-Sensitive Analysis

Given the inherent complexity in precisely identifying the ground truth of selective object-sensitive analysis, this section introduces an approximated upper bound of the ground truth, derived from pointer flows associated with field accesses. We then presents our methodology to access this upper bound of ground truth under context-insensitive analysis.

4.1 Overview of Methodology

This subsection first presents key insights into how we approximate the ground truth, and then outlines our methodology for the effective and efficient selection of target objects.

A primary impact of context sensitivity on objects lies in the replication of their field pointers. If an object satisfies the conditions of Definition 3.4, thereby qualifying as ground truth, then its field pointer replicas across different contexts should effectively prevent the merging of distinct pointer flows. Consequently, determining whether an object benefits from context sensitivity requires analysis on the pointer flows associated with its fields.

Figure 4 presents four distinct fragments of PFGs, each independently illustrating how a heap object h propagates pointer flows through its field f under different circumstances. To maintain clarity and focus on field-related flows, unrelated pointer flows are omitted. In these fragments, y and z denote variables, $h.f$ represents the field f of object h, $c_y^1, c_y^2, c_h^1, c_h^2, c_z^1, c_z^2$ are contexts, and o_1, o_2, o are objects propagated along the pointer flows. For example, in Case 1, two objects o_1, o_2 enter from the left side of the box, propagated along the PFG edges, and eventually reach the right side, remaining separated throughout. Each case captures a different structural pattern of pointer flows. Among them, only Case 1 demonstrates a scenario where the heap object h contributes to improving precision under context sensitivity, which arises from its two key properties: (1) the presence of multiple predecessors receiving distinct pointer flows, and (2) the maintenance of separation between these distinct in-flows during propagation through field pointers. In contrast, field pointers of object h in Case 3 have multiple predecessors that only receive a single identical pointer flow, making differentiation unnecessary. The other two cases exhibit full connection either before or after the field pointers, merging distinct pointer flows at different locations.

Building upon these key insights, we propose a theoretical framework that systematically identifies all ground-truth objects defined in Definition 3.4. First, Theorem 4.1 enables us to transform all pointer flows that influence analysis precision into field-related pointer flows—i.e., the structures depicted in Figure 4—ensuring a universal formulation under object-sensitive pointer analysis (Section 4.2). Next, we derive the necessary conditions under context sensitivity for ground truth (Definition 4.6, termed *Precision-Relevant* objects) and establish their role as an upper bound in Theorem 4.2 (Section 4.3). Finally, leveraging these conditions, we define actionable selection rules under context insensitivity (Definition 4.8) and prove their correctness in Theorem 4.3, ensuring the inclusion of all *Precision-Relevant* objects. For completeness, all detailed formal proofs are provided in the supplemental materials[1] [46], while the main text contains concise proof sketches to aid intuition.

4.2 Targeting Field Accessing during Identification

Prior to presenting our upper-bound approximation of the ground truth, it is important to underscore that context sensitivity distinguishes pointer flows based on calling contexts, and this distinction is primarily introduced by operations that cause context transitions—namely, method calls (**[Method-Call]**) and field accesses (**[Field-Load]** and **[Field-Store]**). The PFG edges built by these operations connect source and destination pointers potentially belonging to different contexts, as demonstrated in Figure 3. Consequently, they play a central role in distinguishing pointer flows under context-sensitive analysis. In contrast, object allocations (**[New]**) and direct assignments (**[Assign]**) are strictly intra-contextual operations, meaning they operate entirely within a single calling context and thus do not contribute to cross-context pointer flow distinctions.

Furthermore, under k-object-sensitive pointer analysis, the influence of pointer flows introduced by **[Method-Call]** on analysis precision can be evaluated analogously to those arising from **[Field-Load]** and **[Field-Store]**. This equivalence is formally established by Theorem 4.1, which allows us to uniformly model pointer flows resulting from method calls in the same way as field accesses. Consequently, the subsequent discussion can focus solely on **[Field-Load]** and **[Field-Store]** operations, without losing generality.

Theorem 4.1. ***(Equivalence of Field and Variable Pointers under Object-Sensitive Pointer Analysis).*** *Under k-object-sensitive pointer analysis, consider (1) a field f of a heap object h, and (2) a local variable v in a method m invoked on h. Then f and v are structurally equivalent in terms of context representation.*

[1] Also available at https://github.com/SEG-DENSE/MOON/blob/main/supplementalMaterials/Supplemental-Material.pdf

Proof Sketch. Given a k-object-sensitive analysis, heap object contexts are represented using sequences of allocation sites with a maximum length of $k-1$, whereas method and variable contexts are tracked with sequences of up to k allocation sites. Assume object h has a context $c_h = [o_1, \ldots, o_l]$, where $l \leq k-1$, according to the rules of object sensitivity, the context of a method m invoked on h is $c_m = [o_1, ..., o_l, h]$. Consequently, a contextual variable v in m is represented as $([o_1, ..., o_l, h], v)$. Similarly, a contextual field pointer for f in h is represented as $([o_1, ..., o_l], h.f)$. If we reinterpret h as an additional context element of field f, we can equivalently represent it as $([o_1, ..., o_l, h], f)$. (Note that the new context has a maximum length of k, consistent with the definition of k-object-sensitivity.) This transformation reveals that the contextual representations of field pointers and local variables take a same structural form under object-sensitive analysis. Thus, it establishes an equivalence between these two types of pointers. □

This theorem allows us to reinterpret variables such as parameters and return variables of a method m as special fields of its receiver objects. Consequently, the edges $(c, a) \rightarrow (c^{m'}, p_{m'})$ and $(c^{m'}, r_{m'}) \rightarrow (c, r)$ constructed by the **[Method-Call]** rule can be viewed as conceptually constructed by the **[Field-Store]** and the **[Field-Load]** rules, respectively.

4.3 Precision-Relevant Objects under Object Sensitivity

Theorem 4.1 establishes that all pointer flows affecting analysis precision can be transformed into field-related pointer flows. Building on this theorem, this section focuses exclusively on field-related pointer flows to derive the necessary conditions for an object to be precision-relevant.

As introduced informally in Section 4.1, two key conditions must be satisfied for an object to improve precision under context sensitivity:

Observation 1. *it must have multiple predecessors (for one of its fields) capable of receiving distinct pointer flows.*

Observation 2. *it must preserve the separation of these distinct pointer flows after propagation through its field pointers.*

Guided by these observations, we now define such selection conditions that govern precision improvements after presenting critical pointer structures in PFGs.

Definition 4.1. ***(Predecessor, Successor, and Base variables of Instance Fields)**. Given an object h and a field f of h, the predecessor, successor and base variables of $h.f$ on PFGs are defined as:*

$$\textsc{Pre}(h,f) := \bigcup_{\substack{(x.f = y) \in S, \\ h \in pt(x)}} \{y\} \quad \textsc{Suc}(h,f) := \bigcup_{\substack{(z = x.f) \in L, \\ h \in pt(x)}} \{z\} \quad \textsc{Base}(h,f) := \bigcup_{\substack{(x.f = y) \in S, h \in pt(x) \\ (z = x.f) \in L, h \in pt(x)}} \{x\}$$

where L and S denote the sets of field load and store statements, respectively.

From the rules **[Field-Store]** and **[Field-Load]** defined in Figure 3, we can derive that on PFGs, the predecessors and successors of instance fields are Pre and Suc variables, respectively. These key nodes on PFGs form the basis for a sequence of definitions below on $G_{\mathbb{O}}$, i.e., the PFG of conventional object-sensitive analysis as specified in Definition 3.3.

Definition 4.2. ***(Full and Partial Connection)**. In a conventional object-sensitive PFG $G_{\mathbb{O}}$, $\bowtie$ and $\not\bowtie$ are predicates to judge whether two pointers s and t are fully or partially connected under different contexts.*

$$s \bowtie t := \forall c_s, c_t \in \mathbb{C} : (c_s, s) \rightsquigarrow (c_t, t)$$
$$s \not\bowtie t := \exists c_s, c_t, c_s', c_t' \in \mathbb{C} : (c_s, s) \rightsquigarrow (c_t, t) \wedge (c_s', s) \not\rightsquigarrow (c_t', t)$$

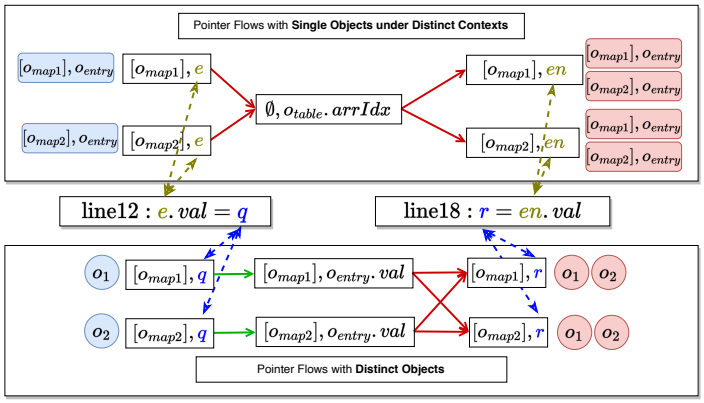


Fig. 5. An illustrative example of precision loss due to ill-separation of pointer flows defined by DistCtx based on the code snippet in Figure 1.

Definition 4.2 characterizes the key connection between two pointers in a PFG. A crucial property of such connection is that when two pointers x and y satisfy $x \bowtie y$, i.e., every contextual pointer of y is reachable from every contextual pointer of x in a $G_{\mathbb{O}}$, the contexts of both x and y become redundant for distinguishing pointer flows from x to y. As a result, these contexts can be safely eliminated without precision loss since they do not affect reachability in PFGs. Consequently, for a given field f of object h, if there exists only a variable $s \in \textsc{Pre}(h, f)$ such that $s \bowtie h.f$ or a variable $t \in \textsc{Suc}(h, f)$ such that $h.f \bowtie t$, then the contexts of object h are considered redundant. This scenario directly corresponds to cases 2 and 4 in Figure 4, where context sensitivity fails to separate distinct pointer flows. Observation 2 states that an object must preserve the separation of distinct pointer flows through its field pointers, and this separation is possible only when there is *partial* (rather than complete) connection around field pointers. This rationale underpins Observation 2 and we formally define it in Definition 4.3.

Definition 4.3. *(**Instance Fields with Partial Connection**). Given a conventional object-sensitive PFG $G_{\mathbb{O}}$ and a field f of object h, Partial is a predicate to judge whether the predecessors and successors of $h.f$ are both partially reachable from or to $h.f$:*

$$\textsc{Partial}(h, f) := \exists p \in \textsc{Pre}(h, f), s \in \textsc{Suc}(h, f) : \quad p \not\bowtie h.f \wedge h.f \not\bowtie s$$

Definition 4.4. *(**Predecessors with Distinct Objects**). Given a conventional object-sensitive PFG $G_{\mathbb{O}}$ and a field f of object h, DistObj is a predicate to judge whether there are predecessors of $h.f$ possessing distinct points-to sets. (Note that y and y' can be an identical variable.)*

$$\textsc{DistObj}(h, f) := \exists y \not\bowtie h.f, y' \in \textsc{Pre}(h, f), c_y, c'_y \in \mathbb{C} : pt(c_y, y) \neq pt(c'_y, y')$$

Definition 4.4 is formulated based on Observation 1, however, it does not fully capture all distinguishable pointer flows, as it imposes no constraints on the contexts of objects along these flows. In practice, when same objects have distinct contexts, these flows may also become distinguishable, which we describe as DistCtx in Definition 4.5. In other words, distinguishable pointer flows should include not only those arising from DistObj, but also those involving objects assigned distinct contexts that are relevant to analysis precision. Figure 5 illustrates this issue using the motivating example from Figure 1. It highlights how the ill-separation of a pointer flow described by DistCtx can lead to the inadequate separation of a pointer flow characterized by DistObj, which finally causes precision loss. Specifically, when object o_{table} is analyzed context-insensitively, two contextual objects, $([o_{map1}], o_{entry})$ and $([o_{map2}], o_{entry})$, are merged at the instance field

$(\emptyset, o_{table}.arrIdx)$, which are then both propagated to the contextual pointers $([o_{map1}], en)$ and $([o_{map2}], en)$, as shown in the upper part of Figure 5. As a consequence, when the field-load statement line 18: $r = en.val$ is processed during object-sensitive analysis, the **[Field-Load]** rule in Figure 3 dictates that $o_{entry}.val \bowtie r$, causing the merging of the pointer flows characterized by DistObj. This merging leads to precision loss, as illustrated in the lower part of Figure 5. Given the critical impact of DistCtx on DistObj, we now formally define *Precision-Relevant* objects.

Definition 4.5. ***(Predecessors with Objects under Distinct Contexts)***. DistCtx *is a predicate to judge whether two flows are distinguishable due to different object contexts. (Note that y and y' can be an identical variable.)*

$$\textsc{DistCtx}(o, h, f) := \begin{array}{c} \exists y \not\bowtie h.f, y' \in \textsc{Pre}(h, f), \exists c_o, c_y, c'_y \in \mathbb{C} : \\ (c_o, o) \in \hat{pt}(c_y, y) \wedge (c_o, o) \notin \hat{pt}(c'_y, y') \end{array}$$

Definition 4.6. ***(Precision-Relevant Objects)***. *The predicate* PR *determines whether an object h is relevant to precision improvement under context sensitivity, with an integer parameter l represents the remaining context-length limit, and the adjusted limit l' is defined as $l' = l - 1$ if h allocates o, and $l' = l$ otherwise.*

$$\textsc{PR}(h, l) := \exists f \in \mathbb{F} \; of \; h : \underbrace{\begin{pmatrix} \textsc{DistObj}(h, f) \vee \\ (\exists y \in \textsc{Pre}(h, f), \exists o \in pt(y) : \\ \textsc{PR}(o, l') \wedge \textsc{DistCtx}(o, h, f) \wedge l' > 0) \end{pmatrix}}_{Distinguishable\ Flows} \wedge \textsc{Partial}(h, f)$$

Definition 4.6 builds directly upon Observation 1 and 2—namely, the presence of distinguishable pointer flows and the existence of partial reachability. In addition to these foundational observations, the definition introduces the notion of *context length limiting*, which explicitly considers the length of object contexts, ensuring contexts of object h meaningfully participate in distinguishable flows to bring potential precision gain. To illustrate this, consider an recursion unfolding process: $[\textsc{PR}(h, l), \ldots, \textsc{PR}(o'', l'')]$, and when the context limit is exhausted at the end of the chain, i.e., $l'' \leq 0$, contexts of h are invisible to o'', making contexts of h redundant to further distinguish the pointer flows. This condition arises frequently in real-world code due to the widespread use of deeply nested data structures. Consider the code in Figure 1, where there is a commonly used data class of **HashMap**, which further encapsulates **Entry** class. In this code example, one key object responsible for distinguishing pointer flows is o_{entry}, allocated at line 10. Under 2-object-sensitive analysis, where the object context depth is limited to 1, due to the depth constraint, the contexts of objects of **HashMap** class are invisible to o_{entry}. As a result, the contexts of o_{map1} and o_{map2}, if present, become redundant. Such cases are prevalent in real-world programs due to the widespread use of encapsulation and the inherent depth limitations of object contexts, especially within container classes in the List, Map, and Set hierarchies.

Besides the context-length limitation, the remaining conditions in Definition 4.6 also serve to ensure flow distinguishability:

- $\textsc{DistObj}(h, f)$ checks whether there exist distinct objects;
- $\textsc{PR}(o, l') \wedge \textsc{DistCtx}(o, h, f)$ checks if o itself is *Precision-Relevant*, meaning its contexts are worth distinguishing, and checks the existence of multiple distinct contexts of o.

These checks precisely capture the essential context dependencies among objects and are able to identify and eliminate more redundant object contexts at theoretical level, resulting in significant performance gains for selective pointer analysis without sacrificing precision. In the meantime, the over-approximation introduced in Definition 4.6, which is primarily due to the predicate Partial, is negligible. In rare cases, the following holds: $\textsc{Partial}(h, f) \wedge (\forall p \in \textsc{Pre}(h, f), s \in \textsc{Suc}(h, f) : p \bowtie s)$.

In such cases, the contexts of h are redundant, as they fail to distinguish any pointer flows. However, these cases are exceedingly uncommon in real-world programs. In practice, the connectivity of instance fields is typically either fully connected or completely isolated, making $\textsc{Partial}(h, f)$ a highly precise over-approximation, which is strongly supported by our experimental results.

Theorem 4.2. *The ground truth of selective context-sensitive objects must be precision-relevant:* $\text{Precision} \uparrow (h) \implies \text{PR}(h, k-1)$, *where* $k-1$ *is the length limit of object contexts.*

Proof Sketch. We prove this theorem by contradiction, i.e., there exists no such field fulfilling the inner conditions in Definition 4.2 while improving precision. We discuss each case respectively.

- **Not Partial:** By contradiction, assume a field f and $\neg\textsc{Partial}(h, f) \iff x \bowtie h.f \vee h.f \bowtie z \implies x \bowtie z$, which implies that even if all the contexts of h are removed and there is only $h.f$ itself, the reachability relations between the predecessors and successors of $h.f$ remain unchanged, i.e., $x \bowtie z$ holds still. Consequently, the contexts of h do not contribute to distinguishing pointer flows across different contexts, i.e., $\neg\text{Precision} \uparrow (h)$.
- **Flow Not Distinguishable:** By contradiction, we have:

$$\neg\textsc{DistObj}(h, f) \wedge (\forall y \in \textsc{Pre}(h, f), \forall o \in pt(y) : \neg\text{PR}(o, l') \vee \neg\textsc{DistCtx}(o, h, f) \vee l' \leq 0).$$

  We discuss each sub-condition below.
  - $\neg\textsc{DistObj}$: By contradiction, for a field f, we have the predecessors of $h.f$ possessing identical objects, making contexts of h redundant for these objects.
  - $\neg\textsc{DistCtx}$: By contradiction, for a field f, we have the predecessors of $h.f$ possessing identical contextual objects, making contexts of h redundant for these contextual objects.
  - $l' \leq 0$: By contradiction, we have the context length limit of o is exceeded. As a result, the contexts of h are invisible to o, making contexts of h redundant for distinguishing the contextual objects of o.
  - $\neg\text{PR}(o, l')$: By contradiction, we have for a field f, every object o pointed to by $y \in \textsc{Pre}(h, f)$ is not *Precision-Relevant* under context limit l'.
    - **Base:** o is not *Precision-Relevant* due to $\neg\textsc{DistObj} \wedge (\neg\textsc{DistCtx} \vee l' \leq 0)$. In such scenarios, we have already shown that the contexts of o are redundant for distinguishing pointer flows, implying that there is no need to differentiate the contextual replicas of o.
    - **Recursive:** o is not *Precision-Relevant* due to $\neg\textsc{DistObj} \wedge \neg\text{PR}$. This results in a recursive situation. However, by unrolling this recursion, we eventually reach one of the previously analyzed cases, which all establish that the contexts of o are redundant for distinguishing pointer flows.

  Since all the conditions identically indicate the contexts of h are redundant, we can derive that $\neg\text{Precision} \uparrow (h)$. □

4.4 Identifying Precision-Relevant Objects under Context Insensitivity

In the previous subsection, we formalized the *Precision-Relevant* objects, an upper-bound approximation for the ground truth in a context-sensitive PFG. While we have proved that *Precision-Relevant* objects definitely cover all the objects that should be applied with context sensitivity for ensuring precision, unfortunately, applying this condition for selection faces a significant challenge: systematically enumerating all *Precision-Relevant* objects still necessitates a time-consuming object-sensitive analysis, which inherently contradicts the efficiency objectives of selective object-sensitive analysis. Hence, to bridge the gap, this subsection further decomposes the definition of *Precision-Relevant* objects for effective identification based on a graph called *Variable Flow Graph (VFG)*, another kind of directed graphs able to indicate points-to relations in a context-insensitive manner.

$$\frac{i : x = new\ T()}{o_i \to x}\ [\textbf{New}]_{\mathcal{G}} \qquad \frac{x = y}{y \to x}\ [\textbf{Assign}]_{\mathcal{G}}$$

$$\frac{x.f = y}{y \to x}\ [\textbf{Field-Store}]_{\mathcal{G}} \qquad \frac{y = x.f}{x \to y,\ y \to x}\ [\textbf{Field-Load}]_{\mathcal{G}} \qquad \frac{r = x.m'(a) \text{ in } m}{a \to x, x \to r, this_{m'} \to p_{m'}, r_{m'} \to this_{m'}}\ [\textbf{Method-Call}]_{\mathcal{G}}$$

Fig. 6. Rules of construction of Variable Flow Graph, with the edges different from PFG colored as Blue

Definition 4.7. ***(Variable Flow Graph (VFG))***. *A variable flow graph is a directed graph* $\mathcal{G} = (\mathcal{N}, \mathcal{E})$*, where* $\mathcal{N} \subseteq (\mathbb{V} \cup \mathbb{O}), \mathcal{E} \subseteq \mathcal{N} \times \mathcal{N}$.

A formal definition of VFG is given in Definition 4.7, while Figure 6 outlines all the rules for constructing a VFG. Similar with a context-insensitive PFG, a VFG does not differentiate contexts, whereas the major difference lies in how pointer flows involving field operations and method calls are represented. Unlike a PFG, which models these flows as edges directly connecting real pointers, a VFG models them by edges between the variables associated with these operations. As a result, edges in PFG involving field operations and method calls can be replaced with fewer, more direct variable-to-variable edges. This transformation makes VFG purely intra-procedural, while still conservatively capture the core pointer flows for analysis purposes. Such optimization downgrades the density of the constructed graphs and reduces the cost of a pre-analysis, which is primarily implemented as graph traversal and necessary for identifying *Precision-Relevant* objects. It is worth noting that, to guarantee traversal completeness, we introduce an additional edge in rule $[\textbf{Field-Load}]_{\mathcal{G}}$. Since a field-load statement typically corresponds to a preceding field-store operation, we add an auxiliary edge $y \to x$ to represent a potential field-store statement, which may appear elsewhere in the program. This prevents potential traversal failures in the VFG due to the missing of field-store statements in the current method, ensuring robust identification of *Precision-Relevant* objects.

Analogous to the definition of *Precision-Relevant* objects based on reachability in context-sensitive PFGs, we introduce a further over-approximation, termed *Applied Precision-Relevant Objects* based on reachability in VFG, as formally defined in Definition 4.8. As outlined in Definition 4.8, we decompose the definition of *Precision-Relevant* objects into four sub-conditions, namely $\overline{\textsc{Partial}}$, $\overline{\textsc{DistObj1}}$, $\overline{\textsc{DistObj2}}$ and $\overline{\textsc{DistCtx}}$, where each sub-condition serves as a necessary condition for the corresponding parts in Definition 4.6. The soundness of this over-approximation is guaranteed by Theorem 4.3, which establishes the implication $\text{PR}(h, k-1) \implies \overline{\text{PR}}(h, k-1)$.

Definition 4.8. ***(Applied Precision-Relevant Objects)***. $\overline{\text{PR}}$ *is a predicate to judge under context insensitivity whether an object is precision-relevant:*

$$\begin{aligned}
\overline{\text{PR}}(h, l) := \quad & \exists f\ of\ h : (\overline{\textsc{DistObj1}} \vee \overline{\textsc{DistObj2}} \vee \overline{\textsc{DistCtx}}) \wedge \overline{\textsc{Partial}} \quad \textbf{\textit{where}} \\
& \overline{\textsc{DistObj1}} = \exists y_1, y_2 \in \textsc{Pre}(h, f) : pt(y_1) \neq pt(y_2) \\
& \overline{\textsc{DistObj2}} = \exists y \in \textsc{Pre}(h, f) : this_m \rightsquigarrow_{\mathcal{G}} y \vee (\exists o \in \mathbb{O} : o \rightsquigarrow_{\mathcal{G}} y) \\
& \overline{\textsc{DistCtx}} = \exists y \in \textsc{Pre}(h, f), \exists o \in \mathbb{O} : o \rightsquigarrow_{\mathcal{G}} y \wedge l' > 0 \wedge \overline{\text{PR}}(o, l') \quad \textbf{\textit{where}} \\
& \qquad\qquad l' = \textbf{\textit{if}}\ h\ allocates\ o\ \textbf{\textit{then}}\ l - 1\ \textbf{\textit{else}}\ l \\
& \overline{\textsc{Partial}} = h \rightsquigarrow_{\mathcal{G}} this_m \vee \exists x \in \textsc{Base}(h, f) : h \rightsquigarrow_{\mathcal{G}} x
\end{aligned}$$

$this_m$ denotes the "this" variable of a given method m, and pt is the result of context-insensitive analysis.

Notably, the "this" variable of a method plays a crucial role in satisfying the conditions $\overline{\textsc{Partial}}$ and $\overline{\textsc{DistObj2}}$ in Definition 4.8. This is because, under object sensitivity, "this" always receives distinct contextual objects across different calling contexts. Consequently, "this" can naturally

separate the pointer flows from the caller to the callee, making it a practical indicator of well-separated pointer flows. Consequently, being reachable from "this" on VFG (e.g., $this_m \rightsquigarrow_{\mathcal{G}} y$) is a necessary condition for a pointer to receive distinct points-to sets under different contexts. Similarly, a newly allocated object has to flow into "this" (i.e., $h \rightsquigarrow_{\mathcal{G}} this_m$) in order to have its contextual replicas assigned to distinct pointers.

Theorem 4.3. *A Precision-Relevant object must be an Applied Precision-Relevant object:* $\text{PR}(h, k-1) \implies \overline{\text{PR}}(h, k-1)$, *where* $k-1$ *is the length limit of object contexts.*

To facilitate the proof of Theorem 4.3, we first introduce the following lemmas, whose proofs are given afterwards to maintain the coherence of presentation.

Lemma 4.1. *Given a field load statement* $y = x.f$, *the following predicate is satisfied under object-sensitive analysis:*

$$\hat{pt}(c_1, y) \neq \hat{pt}(c_2, y) \implies \hat{pt}(c_1, x) \neq \hat{pt}(c_2, x)$$

where $c_1, c_2 \in \mathbb{C}$.

Lemma 4.2. *Under object-sensitive analysis, if a variable* y *under method* m *have different points-to sets under different contexts, we have* $\overline{\text{DistObj2}}$ *in* $\overline{\text{PR}}$ *satisfied on VFG* $\mathcal{G}$:

$$\exists c_1, c_2 \in \mathbb{C} : pt(c_1, y) \neq pt(c_2, y) \implies \hat{pt}(c_1, y) \neq \hat{pt}(c_2, y) \implies this_m \rightsquigarrow_{\mathcal{G}} y \vee (\exists o \in \mathbb{O} : o \rightsquigarrow_{\mathcal{G}} y)$$

Lemma 4.3. *Given a field* f *of* h, *if* $\text{Partial}(h, f)$ *is satisfied on* $G_{\mathbb{O}}$, *we have* $\overline{\text{Partial}}$ *satisfied on VFG* $\mathcal{G}$:

$$\text{Partial}(h, f) \implies \overline{\text{Partial}}$$

We now present the proof sketch for Theorem 4.3.

Proof Sketch. Both PR and $\overline{\text{PR}}$ defined in Definitions 4.6 and 4.8 rely on several atomic properties. Therefore, it suffices to prove each of the following atomic properties individually.

$$\text{Partial}(h, f) \implies \overline{\text{Partial}} \tag{1}$$

$$\text{DistObj}(h, f) \vee \left(\begin{array}{c} \exists y \in \text{Pre}(h, f), \exists o \in pt(y) : \\ \text{PR}(o, l') \wedge \text{DistCtx}(o, h, f) \wedge l' > 0 \end{array} \right) \implies \begin{array}{c} \overline{\text{DistObj1}} \vee \overline{\text{DistObj2}} \\ \vee \overline{\text{DistCtx}} \end{array} \tag{2}$$

By Lemma 4.3, we have (1) proven, thus we only need to prove (2).

Since the recursion depth is bounded by l in both definitions, the number of recursive steps applied is the same on both sides. Therefore, the derived value l' is naturally aligned.

According to Definitions 4.4 and 4.5, a difference between the points-to sets of (c_y, y) and (c'_y, y')—either in terms of objects or contexts—is required. This discrepancy implies either the predicate $\overline{\text{DistObj1}}$ holds directly, or it can be over-approximated via the reachability criteria defined by $\overline{\text{DistObj2}}$ or by the context-sensitive distinction $\overline{\text{DistCtx}}$, as established in Lemma 4.2. □

In the following, we present the proof sketches for Lemmas 4.1, 4.2, and 4.3.

Proof Sketch for Lemma 4.1. We prove Lemma 4.1 by contradiction, if $\hat{pt}(c_1, x) = \hat{pt}(c_2, x) = pts_x$, by rule **[Field-Load]** defined in Figure 3, we have $\forall (c_o, o) \in pts_x : (c_o, o.f) \rightarrow (c_1, y) \wedge (c_o, o.f) \rightarrow (c_2, y)$, thus we have $\hat{pt}(c_1, y) = \hat{pt}(c_2, y)$, which leads to contradiction. □

Proof Sketch for Lemma 4.2. Since $pt(c_1, y) \neq pt(c_2, y) \implies \hat{pt}(c_1, y) \neq \hat{pt}(c_2, y)$ is quite obvious, thus we focus on the proof of $\hat{pt}(c_1, y) \neq \hat{pt}(c_2, y) \implies this_m \rightsquigarrow_{\mathcal{G}} y \vee (\exists o \in \mathbb{O} : o \rightsquigarrow_{\mathcal{G}} y)$.

Assume $\hat{pt}(c_1, y) \neq \hat{pt}(c_2, y)$. We analyze the construction rules that could produce an incoming VFG edge $s \rightarrow_{\mathcal{G}} y$. For each rule:

- **[Assign]**$_{\mathcal{G}}$: For statement $y = s$, $\hat{pt}(c_1, y) \neq \hat{pt}(c_2, y)$ naturally implies $\hat{pt}(c_1, s) \neq \hat{pt}(c_2, s)$.

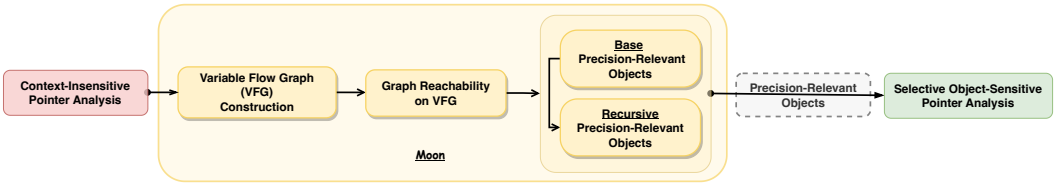


Fig. 7. Overview of Moon.

- **[Field-Load]**$_{\mathcal{G}}$: For statement $y = s.f$, Lemma 4.1 gives $\hat{pt}(c_1, s) \neq \hat{pt}(c_2, s)$. For statement $s = y.f$, it does not affect the points-to set of y thus can be disregarded.
- **[Method-Call]**$_{\mathcal{G}}$: Interprocedural edges are equivalent to field accesses by Theorem 4.1, so $\hat{pt}(c_1, s) \neq \hat{pt}(c_2, s)$.
- **[Field-Store]**$_{\mathcal{G}}$: Edges of the form $s.f = y$ do not affect the points-to set of y, so they are irrelevant.

In all relevant cases, we obtain that $\hat{pt}(c_1, s) \neq \hat{pt}(c_2, s)$, so we recursively apply the same reasoning to s. This recursion must terminate at a root node in the VFG. The only root sources in a method's VFG are either (1) $this_m$: the receiver variable, or (2) an object $o \in \mathbb{O}$: an allocation site. And they inherently carry distinct points-to set across different contexts. Therefore, we have the implication satisfied. □

Proof Sketch for Lemma 4.3. Given a field store statement $x.f = y$, by the context-sensitive analysis rule **[Field-Store]**, we can derive that:

$$(c, y) \rightarrow_{G_{\mathbb{O}}} (c_h, h.f) \iff (c_h, h) \in \hat{pt}(c, x), \text{ where } c, c_h \in \mathbb{C}$$

thus $\textsc{Partial}(h, f)$ requires some (c_h, h) cannot reach (c_x, x) on $G_{\mathbb{O}}$, i.e.,

$$\textsc{Partial}(h, f) \implies (c_h, h) \notin \hat{pt}(c_x, x), \text{ where } c_h, c_x \in \mathbb{C}, x \in \textsc{Base}(h, f)$$

- If we have a base variable x in method m, this leads to $(c_h, h) \notin \hat{pt}(c_x, x) \implies \hat{pt}(c_x, x) \neq \hat{pt}(c'_x, x) \implies h \rightsquigarrow_{\mathcal{G}} x$.
- No base variable within method m directly points to h, yet $\textsc{Partial}(h, f)$ still holds, implying that the distinction between contextual replicas of h is still preserved in m. The only possible source of such a contextual distinction is via $this_m$. Thus $h \rightsquigarrow_{\mathcal{G}} this_m$ must hold. □

5 Implementation

This section presents Moon, our approach designed to efficiently identify *Applied Precision-Relevant* objects following Definition 4.8, which reformulates this problem as sub-problems concerning reachability on VFG.

Figure 7 presents an overview of Moon, which comprises three key components. First, **Variable Flow Graph (VFG) Construction**. The VFG is constructed following the rules outlined in Figure 6, leveraging a rapid yet imprecise context-insensitive pointer analysis as its foundation. Next, **Graph Reachability on VFG**. Forward and backward traversals are performed on the VFG to verify the conditions specified in Definition 4.8. Algorithms 1 and 2 detail the procedures for checking $\overline{\textsc{Partial}}$ and ($\overline{\textsc{DistObj2}}$, $\overline{\textsc{DistCtx}}$), respectively. Notably, $\overline{\textsc{DistObj1}}$ can be directly inferred from the results of context-insensitive analysis and thus does not require a traversal. Eventually, **Selective Object-Sensitive Pointer Analysis** is applied. Context sensitivity is selectively enabled for the objects identified as *Applied Precision-Relevant*, ensuring a balance between precision improvement and analysis efficiency.

We categorized *Applied Precision-Relevant* objects further into two types, i.e. *Base Precision-Relevant* objects and *Recursive Precision-Relevant* objects, and evaluate their contribution to precision

Algorithm 1: Checking of $\overline{\text{PARTIAL}}$ defined in Definition 4.8

Input : $\mathbb{O}$ (Set of all objects)
$\mathcal{G}$ (Variable Flow Graph)
Output: H_p (Set of objects satisfying $\overline{\text{PARTIAL}}$)

```
H_p ← {}
foreach h ∈ 𝕆 do
    reachableNodes ← ForwardTrace(h, 𝒢)
    if reachableNodes contains this_m or Base variables then
        H_p ← H_p ∪ {h}
```

Algorithm 2: Checking of condition $\overline{\text{DISTOBJ2}}$ and $\overline{\text{DISTCTX}}$ defined in Definition 4.8

Input : $\mathbb{O}$ (Set of all objects)
S (Set of Field Store Statements)
$\mathcal{G}$ (Variable Flow Graph)
Output: H_m (Set of objects satisfying $\overline{\text{DISTOBJ2}}$)
M_c (Multi-Map from object satisfying $\overline{\text{DISTCTX}}$ to objects newly allocated)

```
H_m ← {}, M_c ← {}
foreach h ∈ 𝕆 do
    foreach x.f = y ∈ S where h ∈ pt(x) do
        reachableNodes ← BackTrace(y, h, 𝒢)
        if reachableNodes contains this_m then
            H_m ← H_m ∪ {h}
        if reachableNodes contains o ∈ 𝕆 then
            M_c ← M_c ∪ {(h, o)}
```

and efficiency of pointer analysis in Section 6.4. A *Base Precision-Relevant* object satisfies the definition of *Applied Precision-Relevant* objects independently, formalized as $(\overline{\text{DISTOBJ1}} \vee \overline{\text{DISTOBJ2}}) \wedge \overline{\text{PARTIAL}}$. In contrast, a *Recursive Precision-Relevant* object relies on the presence of other *Applied Precision-Relevant* objects to fulfill its definition, characterized by the condition $\overline{\text{DISTCTX}} \wedge \overline{\text{PARTIAL}}$.

6 Evaluation

In this section, we investigate the following research questions for evaluation.

RQ1. Is Moon-guided object-sensitive pointer analysis precise and efficient?

RQ2. How does Moon-guided object-sensitive pointer analysis compare to SOTA alternative techniques?

RQ3. What is the computational overhead of executing Moon compared to SOTA alternative techniques?

RQ4. What is the effect of each type of *Applied Precision-Relevant* objects on selective object-sensitive analysis?

Experimental Implementation. To evaluate our theorem and approach, we implemented a prototype tool, termed Moon, based on Qilin [9], an open-source Java pointer analysis framework. In integrating with Qilin and other frameworks that support selective object-sensitive analysis, Moon configures the identified *Precision-Relevant* objects to be context-sensitive, while treating the remaining objects context-insensitively to achieve selective context sensitivity.

Experimental Settings. All experiments were performed on a desktop equipped with an Intel(R) Core(TM) i9-14900K processor and 192GB of RAM, with a maximum runtime of 1.5 hours per experiment. To ensure a broad range of benchmarks, we selected 30 Java programs, as shown in Table 1. 20 of these benchmarks are frequently referenced in recent studies [7, 8, 19, 25], including 10 JARs from DaCapo-2006 [2], 5 JARs from DaCapo-9.12 and 2 JARs from DaCapo-23.11 [1]. The remaining 10 Java programs, all popular open-source projects, were selected from GitHub in decreasing order of their stars (ranging from 22,000 to 300 stars), with executable jar files provided

Table 1. Benchmark statistics for 30 programs.

Program	Source	#LOC	#Methods	#CallInst	Program	Source	#LOC	#Methods	#CallInst	Program	Source	#LOC	#Methods	#CallInst
antlr	Dacapo-2006	33295	8190	47058	ddjava	GitHub	487622	7249	29452	mindustry	GitHub	318998	6907	28157
avrora	Dacapo-9.12	97133	11373	44036	eclipse	Dacapo-2006	684352	23386	116625	opentelemetry	GitHub	424051	6713	27368
batik	Dacapo-9.12	264397	20994	85033	findbugs	Recent Studies	136631	16774	78935	pmd	Dacapo-2006	212933	12365	54255
biojava	Dacapo-23.11	580432	7910	32226	fop	Dacapo-2006	315825	7997	31634	recaf	GitHub	774345	29038	131990
bloat	Dacapo-2006	43232	9454	45289	h2	Dacapo-9.12	127603	7959	33530	sqlite-jdbc	GitHub	9923	6732	27495
bytecodeviewer	GitHub	1302316	14637	63160	hsqldb	Dacapo-2006	73958	7386	31784	sunflow	Dacapo-9.12	59198	13981	57507
chart	Dacapo-2006	139736	15923	69425	jd	GitHub	135915	17878	78204	tesseract	GitHub	1709887	8313	34315
checkstyle	Recent Studies	141617	12766	61773	JPC	Recent Studies	73906	16138	68969	tomcat	Dacapo-23.11	488604	7388	30204
classyshark	GitHub	236330	10178	41197	luindex	Dacapo-2006	30242	7409	30729	tradebeans	Dacapo-9.12	1305023	8652	36587
dcevm	GitHub	6517	21503	93028	lusearch	Dacapo-2006	30242	8094	33200	xalan	Dacapo-2006	154418	10092	40985

for Moon to take as input. To help readers better understand the scale and complexity of the benchmarks, Table 1 also reports three key metrics for each program, namely lines of code (#LOC), number of reachable methods (#Methods), and number of call instructions (#CallInst). These metrics reflect both the structural size of the program and the potential analysis workload.

Experimental Protocols. In this study, we adopted the k-object-sensitive analysis ($k$obj), which is recognized as a highly practical and precise pointer analysis for Java [26, 27] and widely utilized in recent literature [6, 17, 41]. Specifically, we selected $k = 2$ and $k = 3$, representing 2-object-sensitive (2OBJ) and 3-object-sensitive (3OBJ) analyses, respectively, as the primary object-sensitive pointer analyses for evaluation. Moon was performed on all pointer analyses across all 30 Java programs, with precision loss and consumed analysis time recorded to answer RQ1. Meanwhile, to answer RQ2, four related approaches, Cut-Shortcut [25], the SOTA approach that achieves the effect of context sensitivity without contexts, Zipper [19], the SOTA approach that apply context sensitivity selectively to a subset of methods, as well as Conch [8] and DebloaterX [7], another two SOTA approaches that apply context sensitivity to selected heap objects, were selected as our experimental objects. The pre-analysis time of Moon as well as Zipper, Conch, and DebloaterX was also recorded to answer RQ3 (noting Cut-Shortcut does not require a pre-analysis). In addition, an ablation experiment was conducted on Moon as well as one variant of it, namely: **Moon$_B$**, which uses *Base Precision-Relevant* objects only. This experiment aims to examine the distribution of these object types and their impact on precision and efficiency, addressing RQ4.

Experimental Metrics. In terms of efficiency, analysis **T**ime (metric **T**) and **S**peedup (metric **S**) compared to the baseline $k$obj are used. In terms of precision, four widely recognized clients in the pointer analysis literature [7, 8, 19, 25] were employed, including cast-resolution analysis (metric #**MFC**: number of **M**ay-**F**ail-**C**asts), method reachability analysis (metric #**RM**: number of **R**eachable **M**ethods), devirtualization analysis (metric #**PCS**: number of virtual **C**all **S**ites that remain **P**olymorphic, and call-graph analysis (metric #**CE**: number of **C**all-graph **E**dges). These precision metrics should give a thorough idea of analysis precision for useful clients, whereas in all cases, lower is better. For the precision loss metric, we adopt the same calculation method as defined in the most recent related work [7].

6.1 RQ1: Precision and Efficiency of Moon-Guided Pointer Analysis

Table 2 and Table 3 list the precision and efficiency achieved by Moon-guided pointer analysis (M-2obj and M-3obj) in comparison to 2OBJ and 3OBJ, respectively. Context-insensitive analysis (CI), as well as other four approaches, namely Cut-Shortcut (CSC), Zipper (Z-2obj, Z-3obj), Conch (C-2obj, C-3obj), and DebloaterX (D-2obj, D-3obj), is included for comparison.

6.1.1 How Precise Is Moon-Guided Pointer Analysis Compared to Conventional Analysis? As shown in Tables 2 and 3, Moon demonstrates negligible precision loss compared to both 2OBJ and 3OBJ across all precision metrics on the 30 programs. In comparison to 2OBJ, M-2obj introduce precision

losses of merely 0.1% #**MFC**, 0.2% #**PCS**, 0.1% #**RM**, and 0.0% #**CE** across all 28 of 30 programs that 2OBJ successfully finishes, with an average precision loss of merely 0.1%. To note, among 18 out of the 28 programs that 2OBJ successfully finishes, M-2obj achieves exactly the same #**MFC**, #**PCS**, #**RM**, and #**CE** as 2OBJ, with several well-known Java programs included, such as *tomcat*, *antlr*, and *xalan*, each with over 50k lines of code. These results underscore the noteworthy precision-preserving capability of Moon, even for real-world and large-scale Java programs. For 3OBJ, M-3obj introduces precision losses of 0% #**MFC**, 0.5% #**PCS**, 0.1% #**RM**, and 0.1% #**CE** on 18 programs that 3OBJ successfully finishes, resulting in an average precision loss of merely 0.2%. All precision losses remain below 0.6% across all metrics and programs, with only one exception: a 2.2% loss in #**PCS** on *JPC*. Further analysis indicates that Moon's precision loss is closely tied to the narrowing strategy which we have given detailed explanations in Section 6.1.3.

6.1.2 How Fast Is Moon-Guided Pointer Analysis Compared to Conventional Analysis? In comparison to 2OBJ, M-2obj achieves speedups ranging from 4.0X to 93.1X across 28 programs, with an average speedup of over 37.2X (with time of Oom or Oot as 1.5h). Notably, the highest speedup is observed with *findbugs*, where the conventional 2OBJ analysis requires over 13 minutes, whereas Moon finishes its analysis within 9 seconds. In comparison to 3OBJ on 18 programs, the speedup achieved by M-3obj is even more pronounced, with speedups ranging from 125.8X to 1319.1X and an average speedup of over 382.0X. It is also noteworthy that the efficiency of M-2obj and M-3obj even outperforms CI on 23 programs for 2OBJ and 18 programs for 3OBJ, as colored in red. This superior performance is attributed to the modest set of context-sensitive objects selected by Moon, which significantly reduces both context-sensitive analysis overhead and unnecessary pointer propagation especially on virtual calls. This finding aligns with prior research suggesting that "precision loss may also significantly hamper efficiency" [19, 30]. Specifically, for Moon, the small overhead from selected context-sensitive objects is even effectively offset by the efficiency gains from reduced pointer propagation.

Further evidence supporting the outstanding efficiency of Moon-guided pointer analysis can be seen in the extremely small percentage of *Precision-Relevant* objects identified by Moon, as shown in Figure 8. Across 30 programs, only 1.1%-3.8% and 2.1%-6.1% objects are selected as *Precision-Relevant* by Moon for 2OBJ and 3OBJ, respectively, with an average of 1.6% and 2.9%.

6.1.3 Discussion for Precision Loss Due to Narrowing Strategies in Moon. The primary source of precision loss in Moon stems from its narrowing strategy. Under *Object-Sensitivity*, by Theorem 4.1, a parameter p of method m can be seen as a conceptual field of the receiver objects invoking m. Moon can efficiently handle this by treating method parameters as fields, thereby capturing all *Precision-Relevant* objects. However, it excludes those objects involving pointer flows only related to parameters, without corresponding flows for fields. This exclusion is based on the rarity of such cases in object-oriented programming. Including these objects would significantly impacting the efficiency of pointer analysis.

Despite such narrowing strategies, Moon is still proved to be both precise and efficient: it incurs only negligible precision losses of 0.1% and 0.2%, while achieving 37.2X and 382.0X speedups for 2-object-sensitive and 3-object-sensitive analyses, respectively.

6.2 RQ2: Comparison of Moon and Other Selective Heuristic Approaches

Besides M-2obj and M-3obj, Table 2 and 3 also list the experimental results of Cut-Shortcut, Zipper, Conch, and DebloaterX. Among the other four approaches, DebloaterX achieves the lowest overall precision losses of 0.05% and 0.2% for 2OBJ and 3OBJ, respectively. These figures are slightly lower than the losses observed with Conch, which are 0.3% and 0.3%, and are considerably lower than those reported for Zipper (4.6% and 5.3%) and Cut-Shortcut (62.8% and 66.9%).

Table 2. Precision and Efficiency results of CSC, Z-2obj, C-2obj, D-2obj, and M-2obj, whereas CI and 2OBJ are presented as baselines.

Program	Analysis	T	S	#MFC	#PCS	#RM	#CE	Program	Analysis	T	S	#MFC	#PCS	#RM	#CE	Program	Analysis	T	S	#MFC	#PCS	#RM	#CE
antlr	2obj	26.5	-	510	1631	7806	51264	avrora	2obj	18.8	-	568	933	10940	48361	batik	2obj	810.7	-	2142	4956	20254	107822
	ci	4.8	5.5X	1124	1976	8190	57341		ci	5.5	3.4X	1188	1372	11373	55476		ci	13.3	61.0X	3403	5810	20994	127604
	CSC	5.7	4.6X	812	1896	8153	54460		CSC	6.3	3.0X	882	1283	11282	52535		CSC	18.6	43.6X	2846	5524	20819	121105
	Z-2obj	12.6	2.1X	528	1650	7836	51391		Z-2obj	6.1	3.1X	595	948	10969	48466		Z-2obj	370.6	2.2X	2161	4968	20265	107892
	C-2obj	6.2	4.3X	510	1631	7806	51264		C-2obj	5.7	3.3X	568	933	10940	48361		C-2obj	375.2	2.2X	2142	4956	20254	107822
	D-2obj	5.3	5.0X	510	1631	7806	51264		D-2obj	3.9	4.8X	568	933	10940	48361		D-2obj	245.7	3.3X	2142	4958	20254	107824
	M-2obj	2.4	11.0X	510	1631	7806	51264		M-2obj	2.4	7.8X	568	933	10940	48361		M-2obj	201.7	4.0X	2150	4962	20254	107828
biojava	2obj	13.5	-	436	843	7440	34802	bloat	2obj	352.9	-	1285	1566	9019	56577	bytecode-viewer	2obj	2992.2	-	4064	4626	14210	100280
	ci	4.5	3.0X	1046	1236	7910	41678		ci	5.5	64.5X	2082	2249	9454	67338		ci	9.4	316.6X	4723	5616	14637	110795
	CSC	5.7	2.4X	687	1150	7853	38403		CSC	7.6	46.4X	1594	2166	9414	63909		CSC	21.7	138.0X	4366	5547	14601	106506
	Z-2obj	4.5	3.0X	449	876	7487	34947		Z-2obj	279.7	1.3X	1309	1597	9061	56784		Z-2obj	2121.9	1.4X	4080	4644	14246	100443
	C-2obj	4.6	3.0X	436	843	7440	34802		C-2obj	202.7	1.7X	1285	1566	9019	56577		C-2obj	1662.4	1.8X	4064	4626	14210	100280
	D-2obj	3.3	4.1X	436	843	7440	34802		D-2obj	15.0	23.5X	1285	1566	9019	56577		D-2obj	686.2	4.4X	4064	4626	14210	100280
	M-2obj	1.6	8.2X	436	843	7440	34802		M-2obj	9.4	37.7X	1285	1566	9019	56577		M-2obj	216.5	13.8X	4065	4629	14210	100283
chart	2obj	100.2	-	1337	2027	15149	72650	check-style	2obj	Oom	-	-	-	-	-	classy-shark	2obj	22.3	-	564	1117	9393	44257
	ci	9.2	10.9X	2560	2703	15923	86433		ci	8.2	>657.7X	1939	2760	12766	80169		ci	5.3	4.2X	1323	1682	10178	56231
	CSC	10.4	9.6X	1914	2405	15545	78847		CSC	10.3	>521.7X	1503	2641	12713	74197		CSC	7.1	3.1X	876	1400	9489	48577
	Z-2obj	15.9	6.3X	1376	2045	15234	72989		Z-2obj	1446.3	>3.7X	1132	2249	12342	67170		Z-2obj	8.6	2.6X	581	1124	9412	44332
	C-2obj	42.7	2.3X	1337	2027	15149	72650		C-2obj	3804.2	>1.4X	1109	2216	12307	66970		C-2obj	7.6	2.9X	564	1117	9393	44257
	D-2obj	18.2	5.5X	1337	2027	15149	72658		D-2obj	17.6	>307.7X	1109	2216	12307	66970		D-2obj	4.9	4.6X	564	1117	9393	44257
	M-2obj	7.9	12.6X	1337	2027	15149	72658		M-2obj	7.8	>695.9X	1110	2216	12307	66970		M-2obj	2.5	8.7X	564	1117	9393	44257
dcevm	2obj	374.1	-	2225	4417	20531	100557	ddjava	2obj	10.9	-	367	815	6840	32248	eclipse	2obj	Oom	-	-	-	-	-
	ci	13.5	27.7X	3593	5669	21503	124314		ci	4.1	2.7X	880	1177	7249	38462		ci	18.8	>287.7X	5090	10672	23386	182975
	CSC	15.2	24.6X	2917	4932	20958	108001		CSC	4.8	2.3X	618	1093	7205	35797		CSC	20.7	>261.4X	4282	10250	22977	172521
	Z-2obj	143.0	2.6X	2316	4464	20573	100824		Z-2obj	5.0	2.2X	384	827	6855	32316		Z-2obj	Oom	-	-	-	-	-
	C-2obj	186.5	2.0X	2227	4425	20532	100594		C-2obj	3.0	3.6X	367	815	6840	32248		C-2obj	1301.6	>4.1X	3612	9703	22627	162654
	D-2obj	165.0	2.3X	2231	4431	20532	100608		D-2obj	2.5	4.3X	367	815	6840	32248		D-2obj	522.0	>10.3X	3612	9703	22627	162654
	M-2obj	63.3	5.9X	2241	4433	20535	100629		M-2obj	1.8	5.9X	367	815	6840	32248		M-2obj	245.5	>22.0X	3612	9703	22627	162659
findbugs	2obj	811.2	-	2013	3508	16264	87808	fop	2obj	10.2	-	395	830	7591	34364	h2	2obj	17.0	-	398	911	7559	36261
	ci	9.7	83.7X	3448	4435	16774	105576		ci	4.0	2.5X	911	1210	7997	40423		ci	4.6	3.7X	943	1274	7959	42799
	CSC	11.6	70.0X	2766	4212	16705	96294		CSC	4.5	2.2X	653	1121	7944	37562		CSC	5.2	3.3X	687	1194	7914	40294
	Z-2obj	132.5	6.1X	2044	3512	16276	87855		Z-2obj	4.5	2.3X	417	852	7621	34496		Z-2obj	5.8	2.9X	423	929	7591	36379
	C-2obj	44.0	18.4X	2014	3508	16264	87808		C-2obj	3.1	3.3X	395	830	7591	34364		C-2obj	4.9	3.5X	398	911	7559	36261
	D-2obj	14.8	54.8X	2014	3508	16264	87808		D-2obj	2.6	3.8X	395	830	7591	34364		D-2obj	3.7	4.6X	398	911	7559	36261
	M-2obj	8.7	93.1X	2017	3511	16268	87838		M-2obj	1.8	5.8X	395	830	7591	34364		M-2obj	1.9	9.1X	398	911	7559	36261
hsqldb	2obj	11.8	-	407	847	6981	34881	jd	2obj	58.0	-	1592	2831	17000	82056	JPC	2obj	65.3	-	1356	4237	15553	81294
	ci	4.4	2.7X	922	1202	7386	41755		ci	10.0	5.8X	2714	3750	17878	95602		ci	10.3	6.3X	2252	4934	16138	94701
	CSC	5.0	2.3X	657	1119	7331	38607		CSC	11.8	4.9X	2127	3278	17354	87467		CSC	11.4	5.7X	1790	4763	16034	87453
	Z-2obj	4.2	2.8X	428	868	7015	35020		Z-2obj	15.7	3.7X	1686	2871	17045	82387		Z-2obj	18.7	3.5X	1382	4274	15582	81489
	C-2obj	3.5	3.4X	407	847	6981	34881		C-2obj	19.8	2.9X	1592	2831	17000	82060		C-2obj	30.1	2.2X	1356	4238	15553	81307
	D-2obj	3.0	3.9X	407	847	6981	34881		D-2obj	19.4	3.0X	1592	2832	17000	82061		D-2obj	27.5	2.4X	1356	4243	15554	81319
	M-2obj	1.8	6.8X	407	847	6981	34881		M-2obj	9.0	6.5X	1597	2832	17000	82061		M-2obj	13.0	5.0X	1360	4243	15554	81321
luindex	2obj	10.7	-	395	923	7019	33587	lusearch	2obj	11.8	-	409	1119	7671	36464	mindustry	2obj	9.3	-	378	778	6497	30844
	ci	3.9	2.7X	920	1283	7409	39677		ci	4.2	2.8X	1032	1490	8094	43014		ci	3.8	2.4X	865	1136	6907	36808
	CSC	4.5	2.4X	641	1201	7357	36882		CSC	4.8	2.5X	724	1411	8053	40001		CSC	4.3	2.2X	622	1051	6862	33970
	Z-2obj	4.1	2.6X	417	944	7048	33718		Z-2obj	4.8	2.5X	434	1143	7700	36598		Z-2obj	4.2	2.2X	398	805	6540	30984
	C-2obj	3.2	3.3X	395	923	7019	33588		C-2obj	3.4	3.5X	409	1119	7671	36464		C-2obj	3.0	3.1X	378	778	6497	30844
	D-2obj	2.7	3.9X	395	923	7019	33588		D-2obj	3.1	3.8X	409	1119	7671	36464		D-2obj	2.1	4.5X	378	778	6497	30844
	M-2obj	1.6	6.9X	395	923	7019	33588		M-2obj	1.6	7.2X	409	1119	7671	36464		M-2obj	1.4	6.7X	378	778	6497	30844
open-telemetry	2obj	9.2	-	351	767	6303	29976	pmd	2obj	32.5	-	1398	2354	11851	59910	recaf	2obj	475.3	-	3247	5701	27823	142265
	ci	3.9	2.4X	825	1117	6713	35944		ci	6.5	5.0X	2265	2950	12365	69505		ci	20.4	23.3X	5127	7402	29038	187415
	CSC	4.2	2.2X	581	1035	6668	33157		CSC	7.6	4.3X	1748	2710	12273	64834		CSC	34.0	14.0X	4138	6702	28300	161535
	Z-2obj	3.5	2.6X	365	789	6335	30086		Z-2obj	14.5	2.2X	1428	2375	11882	60047		Z-2obj	166.4	2.9X	3378	5732	27857	142450
	C-2obj	2.8	3.3X	351	767	6303	29976		C-2obj	17.4	1.9X	1398	2354	11851	59910		C-2obj	203.0	2.3X	3249	5716	27823	142326
	D-2obj	2.3	4.0X	351	767	6303	29976		D-2obj	6.5	5.0X	1398	2354	11851	59910		D-2obj	161.8	2.9X	3249	5722	27823	142332
	M-2obj	1.3	7.0X	351	767	6303	29976		M-2obj	4.3	7.5X	1398	2354	11851	59910		M-2obj	104.6	4.5X	3257	5722	27826	142365
sqlite-jdbc	2obj	8.7	-	351	768	6322	30094	sunflow	2obj	35.7	-	1172	1965	13313	61168	tesseract	2obj	11.7	-	399	843	7743	37267
	ci	3.8	2.3X	826	1118	6732	36062		ci	8.2	4.3X	2146	2553	13981	72716		ci	4.6	2.5X	1013	1291	8313	45068
	CSC	4.4	2.0X	581	1036	6687	33275		CSC	9.0	4.0X	1660	2359	13797	67237		CSC	6.1	1.9X	747	1216	8270	41968
	Z-2obj	4.3	2.0X	365	790	6354	30204		Z-2obj	9.2	3.9X	1215	1988	13401	61537		Z-2obj	4.5	2.6X	421	867	7775	37383
	C-2obj	2.7	3.2X	351	768	6322	30094		C-2obj	10.5	3.4X	1186	1994	13346	61442		C-2obj	3.1	3.8X	399	843	7743	37267
	D-2obj	2.2	3.9X	351	768	6322	30094		D-2obj	7.7	4.7X	1172	1965	13314	61173		D-2obj	2.7	4.3X	399	843	7743	37267
	M-2obj	1.4	6.4X	351	768	6322	30094		M-2obj	3.9	9.1X	1172	1965	13314	61173		M-2obj	1.6	7.5X	399	843	7743	37267
tomcat	2obj	16.8	-	402	871	6978	33095	trade-beans	2obj	26.7	-	466	967	8021	39111	xalan	2obj	372.9	-	592	1643	9666	46782
	ci	4.0	4.1X	931	1320	7388	39513		ci	4.7	5.7X	1095	1420	8652	47885		ci	5.2	71.3X	1298	2084	10092	54001
	CSC	4.6	3.6X	665	1243	7340	37125		CSC	5.5	4.9X	786	1314	8546	44610		CSC	6.0	61.6X	884	1967	10034	50747
	Z-2obj	5.3	3.2X	421	895	7022	33248		Z-2obj	11.1	2.4X	501	1017	8213	40042		Z-2obj	177.7	2.1X	609	1665	9697	46916
	C-2obj	4.0	4.2X	402	871	6978	33095		C-2obj	6.7	4.0X	481	999	8065	39461		C-2obj	209.0	1.8X	592	1643	9666	46782
	D-2obj	2.6	6.5X	402	871	6978	33095		D-2obj	4.3	6.3X	466	967	8021	39111		D-2obj	14.1	26.5X	592	1643	9666	46782
	M-2obj	1.7	10.1X	402	871	6978	33095		M-2obj	2.6	10.4X	466	967	8021	39111		M-2obj	5.1	73.1X	592	1643	9666	46782

Table 3. Precision and Efficiency results of CSC, Z-3obj, C-3obj, D-3obj, and M-3obj, whereas CI and 3OBJ are presented as baselines.

Program	Analysis	T	S	#MFC	#PCS	#RM	#CE	Program	Analysis	T	S	#MFC	#PCS	#RM	#CE	Program	Analysis	T	S	#MFC	#PCS	#RM	#CE
antlr	3obj	526.2	-	450	1624	7805	51237	avrora	3obj	2592.0	-	496	929	10939	48330	batik	3obj	OOM	-	-	-	-	-
	ci	4.8	108.7X	1124	1976	8190	57341		ci	5.5	470.4X	1188	1372	11373	55476		ci	13.3	>406.3X	3403	5810	20994	127604
	CSC	5.7	92.0X	812	1896	8153	54460		CSC	6.3	409.5X	882	1283	11282	52535		CSC	18.6	>290.2X	2846	5524	20819	121105
	Z-3obj	225.0	2.3X	473	1643	7835	51364		Z-3obj	878.4	3.0X	536	944	10969	48439		Z-3obj	OOT	-	-	-	-	-
	C-3obj	119.6	4.4X	450	1624	7805	51237		C-3obj	578.7	4.5X	496	929	10939	48330		C-3obj	OOM	-	-	-	-	-
	D-3obj	7.8	67.8X	450	1627	7805	51240		D-3obj	6.1	424.2X	496	931	10939	48332		D-3obj	OOM	-	-	-	-	-
	M-3obj	2.7	197.8X	450	1626	7805	51239		M-3obj	2.9	887.7X	496	930	10939	48336		M-3obj	745.5	>7.2X	2044	4930	20243	106779
biojava	3obj	1053.9	-	347	837	7439	34756	bloat	3obj	OOT	-	-	-	-	-	bytecode-viewer	3obj	OOT	-	-	-	-	-
	ci	4.5	232.7X	1046	1236	7910	41678		ci	5.5	>987.2X	2082	2249	9454	67338		ci	9.4	>571.4X	4723	5616	14637	110795
	CSC	5.7	186.5X	687	1150	7853	38403		CSC	7.6	>710.5X	1594	2166	9414	63909		CSC	21.7	>249.0X	4366	5547	14601	106506
	Z-3obj	202.8	5.2X	369	870	7486	34901		Z-3obj	4772.6	>1.1X	1215	1583	9045	56555		Z-3obj	OOT	-	-	-	-	-
	C-3obj	184.3	5.7X	347	837	7439	34756		C-3obj	1447.0	>3.7X	1188	1552	9003	56348		C-3obj	OOT	-	-	-	-	-
	D-3obj	5.7	185.9X	347	839	7439	34758		D-3obj	19.6	>275.8X	1188	1555	9003	56351		D-3obj	OOT	-	-	-	-	-
	M-3obj	1.9	543.3X	347	838	7439	34757		M-3obj	9.4	>572.6X	1188	1554	9003	56350		M-3obj	1825.6	>3.0X	3661	4195	14042	94460
chart	3obj	OOM	-	-	-	-	-	check-style	3obj	OOM	-	-	-	-	-	classy-shark	3obj	3020.7	-	484	1106	9384	44166
	ci	9.2	>587.6X	2560	2703	15923	86433		ci	8.2	>657.7X	1939	2760	12766	80169		ci	5.3	566.7X	1323	1682	10178	56231
	CSC	10.4	>519.7X	1914	2405	15545	78847		CSC	10.3	>521.7X	1503	2641	12713	74197		CSC	7.1	425.4X	876	1400	9489	48577
	Z-3obj	392.5	>13.8X	1280	2021	15204	72544		Z-3obj	OOT	-	-	-	-	-		Z-3obj	1393.7	2.2X	512	1113	9407	44251
	C-3obj	OOT	-	-	-	-	-		C-3obj	OOT	-	-	-	-	-		C-3obj	1046.5	2.9X	484	1106	9384	44166
	D-3obj	152.8	>35.3X	1230	2005	15119	72228		D-3obj	24.1	>224.4X	996	2172	12260	66188		D-3obj	8.3	363.5X	484	1108	9384	44168
	M-3obj	34.1	>158.5X	1230	2004	15119	72227		M-3obj	10.4	>517.2X	1000	2171	12260	66188		M-3obj	3.3	907.1X	484	1107	9386	44176
dcevm	3obj	OOM	-	-	-	-	-	ddjava	3obj	398.8	-	312	810	6840	32226	eclipse	3obj	OOM	-	-	-	-	-
	ci	13.5	>399.4X	3593	5669	21503	124314		ci	4.1	98.0X	880	1177	7249	38462		ci	18.8	>287.7X	5090	10672	23386	182975
	CSC	15.2	>355.0X	2917	4932	20958	108001		CSC	4.8	83.4X	618	1093	7205	35797		CSC	20.7	>261.4X	4282	10250	22977	172521
	Z-3obj	OOM	-	-	-	-	-		Z-3obj	118.3	3.4X	336	823	6855	32290		Z-3obj	OOT	-	-	-	-	-
	C-3obj	OOT	-	-	-	-	-		C-3obj	113.6	3.5X	312	810	6840	32226		C-3obj	OOM	-	-	-	-	-
	D-3obj	1076.8	>5.0X	2131	4396	20493	100083		D-3obj	4.7	84.5X	312	812	6840	32228		D-3obj	OOM	-	-	-	-	-
	M-3obj	474.7	>11.4X	2131	4396	20493	100084		M-3obj	1.9	208.8X	312	812	6840	32229		M-3obj	3691.6	>1.5X	3484	9661	22572	161716
findbugs	3obj	OOM	-	-	-	-	-	fop	3obj	363.4	-	336	824	7591	34344	h2	3obj	1942.4	-	336	907	7557	36232
	ci	9.7	>557.3X	3448	4435	16774	105576		ci	4.0	91.1X	911	1210	7997	40423		ci	4.6	423.2X	943	1274	7959	42799
	CSC	11.6	>465.9X	2766	4212	16705	96294		CSC	4.5	80.0X	653	1121	7944	37562		CSC	5.2	376.4X	687	1194	7914	40294
	Z-3obj	OOT	-	-	-	-	-		Z-3obj	111.9	3.2X	370	846	7621	34476		Z-3obj	641.5	3.0X	373	925	7590	36354
	C-3obj	494.5	>10.9X	1632	3477	16217	86892		C-3obj	102.3	3.6X	336	824	7591	34344		C-3obj	420.7	4.6X	336	907	7557	36232
	D-3obj	18.6	>290.2X	1652	3478	16217	86899		D-3obj	4.5	81.5X	336	826	7591	34346		D-3obj	7.1	274.7X	336	909	7558	36237
	M-3obj	10.3	>521.7X	1652	3478	16219	86915		M-3obj	2.0	180.8X	336	825	7591	34345		M-3obj	2.6	735.8X	336	908	7558	36236
hsqldb	3obj	588.8	-	355	840	6980	34854	jd	3obj	OOM	-	-	-	-	-	JPC	3obj	1438.2	-	1204	4101	15206	79620
	ci	4.4	134.7X	922	1202	7386	41755		ci	10.0	>538.4X	2714	3750	17878	95602		ci	10.3	139.5X	2252	4934	16138	94701
	CSC	5.0	116.8X	657	1119	7331	38607		CSC	11.8	>459.2X	2127	3278	17354	87467		CSC	11.4	125.9X	1790	4763	16034	87453
	Z-3obj	159.7	3.7X	382	861	7014	34993		Z-3obj	239.1	>22.6X	1600	2818	16979	81855		Z-3obj	173.0	8.3X	1234	4139	15233	79803
	C-3obj	155.4	3.8X	355	840	6980	34854		C-3obj	239.2	>22.6X	1491	2751	16875	81318		C-3obj	143.1	10.1X	1204	4102	15206	79633
	D-3obj	5.3	111.9X	355	843	6980	34857		D-3obj	67.1	>80.5X	1491	2754	16875	81326		D-3obj	37.8	38.1X	1204	4105	15207	79641
	M-3obj	2.0	290.0X	355	842	6980	34856		M-3obj	19.2	>281.7X	1491	2754	16875	81327		M-3obj	6.9	208.1X	1204	4119	15209	79684
luindex	3obj	376.2	-	341	916	7018	33560	lusearch	3obj	582.3	-	357	1112	7670	36437	mindustry	3obj	325.0	-	321	771	6495	30815
	ci	3.9	96.5X	920	1283	7409	39677		ci	4.2	139.6X	1032	1490	8094	43014		ci	3.8	85.1X	865	1136	6907	36808
	CSC	4.5	82.9X	641	1201	7357	36882		CSC	4.8	121.6X	724	1411	8053	40001		CSC	4.3	75.2X	622	1051	6862	33970
	Z-3obj	130.8	2.9X	364	937	7047	33691		Z-3obj	172.1	3.4X	382	1136	7699	36571		Z-3obj	103.7	3.1X	352	799	6540	30962
	C-3obj	119.0	3.2X	341	916	7018	33561		C-3obj	145.0	4.0X	357	1112	7670	36437		C-3obj	102.8	3.2X	321	771	6495	30815
	D-3obj	4.8	77.7X	341	919	7018	33564		D-3obj	5.0	116.5X	357	1115	7670	36440		D-3obj	3.6	89.0X	321	773	6495	30817
	M-3obj	1.8	204.4X	341	918	7018	33563		M-3obj	2.7	215.7X	357	1114	7670	36439		M-3obj	1.7	190.1X	321	773	6495	30817
open-telemetry	3obj	325.3	-	296	760	6303	29955	pmd	3obj	872.1	-	1333	2348	11851	59851	recaf	3obj	OOM	-	-	-	-	-
	ci	3.9	83.2X	825	1117	6713	35944		ci	6.5	133.3X	2265	2950	12365	69505		ci	20.4	>265.1X	5127	7402	29038	187415
	CSC	4.2	77.5X	581	1035	6668	33157		CSC	7.6	114.3X	1748	2710	12273	64834		CSC	34.0	>158.6X	4138	6702	28300	161535
	Z-3obj	97.7	3.3X	318	783	6335	30066		Z-3obj	269.9	3.2X	1374	2369	11882	59988		Z-3obj	OOM	-	-	-	-	-
	C-3obj	103.3	3.1X	296	760	6303	29955		C-3obj	155.8	5.6X	1333	2348	11851	59851		C-3obj	OOM	-	-	-	-	-
	D-3obj	4.0	81.5X	296	762	6303	29957		D-3obj	8.7	100.0X	1333	2350	11851	59853		D-3obj	1068.5	>5.1X	3037	5651	27467	140236
	M-3obj	1.6	203.3X	296	762	6303	29957		M-3obj	6.9	125.8X	1333	2349	11851	59852		M-3obj	437.9	>12.3X	3038	5646	27468	140249
sqlite-jdbc	3obj	310.9	-	296	761	6322	30073	sunflow	3obj	OOM	-	-	-	-	-	tesseract	3obj	733.4	-	341	836	7743	37221
	ci	3.8	81.0X	826	1118	6732	36062		ci	8.2	>654.5X	2146	2553	13981	72716		ci	4.6	159.1X	1013	1291	8313	45068
	CSC	4.4	70.3X	581	1036	6687	33275		CSC	9.0	>600.7X	1660	2359	13797	67237		CSC	6.1	120.0X	747	1216	8270	41968
	Z-3obj	95.2	3.3X	318	784	6354	30184		Z-3obj	761.1	>7.1X	1116	1976	13382	61274		Z-3obj	188.7	3.9X	372	861	7775	37339
	C-3obj	101.3	3.1X	296	761	6322	30073		C-3obj	582.4	>9.3X	1083	1976	13325	61154		C-3obj	179.2	4.1X	341	836	7743	37221
	D-3obj	3.8	82.5X	296	763	6322	30075		D-3obj	29.6	>182.7X	1071	1955	13294	60907		D-3obj	4.3	172.2X	341	838	7743	37223
	M-3obj	1.6	196.7X	296	763	6322	30075		M-3obj	5.0	>1086.5X	1071	1954	13294	60919		M-3obj	1.8	409.7X	341	838	7743	37223
tomcat	3obj	687.6	-	345	866	6977	33068	trade-beans	3obj	4379.4	-	401	962	7989	38931	xalan	3obj	OOM	-	-	-	-	-
	ci	4.0	169.8X	931	1320	7388	39513		ci	4.7	927.8X	1095	1420	8652	47885		ci	5.2	>1032.5X	1298	2084	10092	54001
	CSC	4.6	149.1X	665	1243	7340	37125		CSC	5.5	799.2X	786	1314	8546	44610		CSC	6.0	>892.6X	884	1967	10034	50747
	Z-3obj	158.9	4.3X	374	891	7022	33226		Z-3obj	1674.5	2.6X	448	1015	8209	39991		Z-3obj	1528.5	>3.5X	561	1658	9696	46887
	C-3obj	136.8	5.0X	345	866	6977	33068		C-3obj	1144.2	3.8X	417	991	8040	39276		C-3obj	452.8	>11.9X	539	1636	9665	46753
	D-3obj	4.6	149.5X	345	868	6977	33070		D-3obj	7.2	604.9X	401	964	7989	38933		D-3obj	18.4	>293.8X	539	1639	9665	46756
	M-3obj	2.0	350.8X	345	868	6977	33070		M-3obj	3.3	1319.1X	401	963	7991	38941		M-3obj	5.9	>912.2X	539	1638	9665	46755

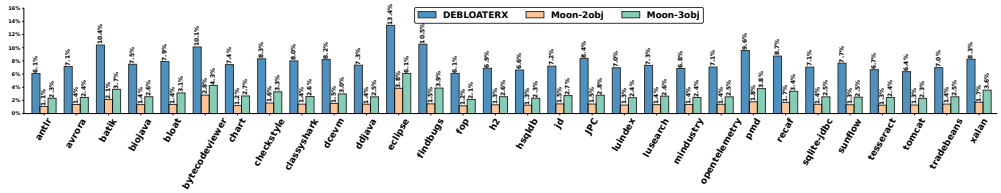
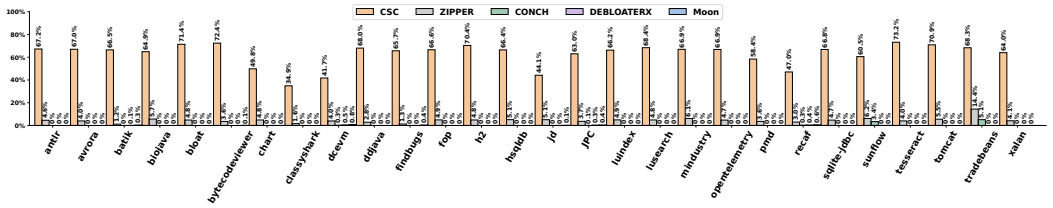
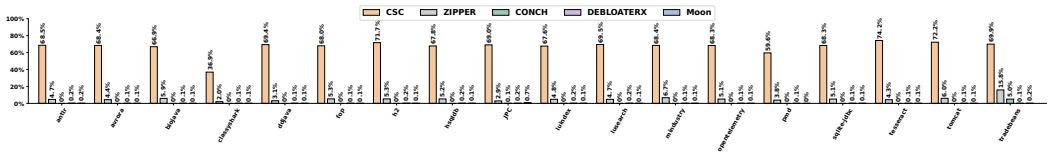


Fig. 8. Context-sensitive objects identified by DebloaterX and Moon (both M-2obj and M-3obj).



(a) Percentages of precision loss of Cut-Shortcut, Z-2obj, C-2obj, D-2obj, and M-2obj.

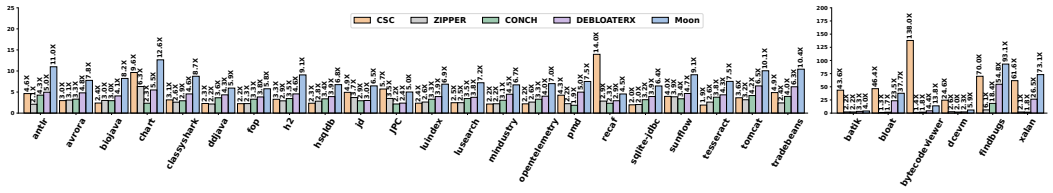


(b) Percentages of precision loss of Cut-Shortcut, Z-3obj, C-3obj, D-3obj, and M-3obj.

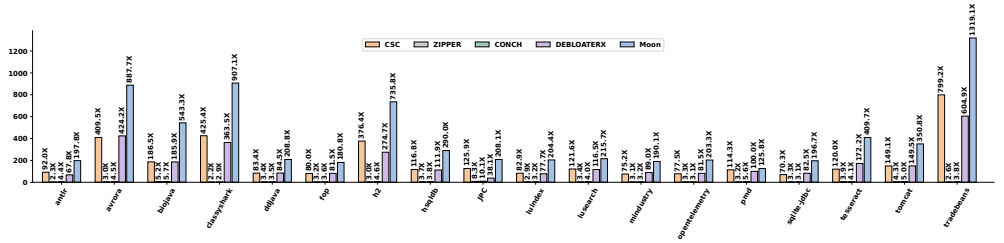
Fig. 9. Percentages of overall precision loss of other four approaches and Moon-guided pointer analysis on 2OBJ (a) and 3OBJ (b) over conventional analysis.

The high precision-persevering capability exhibited by DebloaterX aligns with the experimental results in their research paper, however, our Moon exhibits comparable overall precision losses of 0.1% for 2OBJ and 0.2% for 3OBJ, though with slightly higher precision losses on several programs such as *decevm*, *findbugs*, and *recaf*, etc, as indicated in Figure 9a and 9b. Nevertheless, in comparison to the other three approaches, Moon demonstrates greater precision than Conch and significantly and comprehensively higher precision than both Zipper and Cut-Shortcut. In addition, on another 17 programs including *antlr*, *checkstyle*, and *xalan* for 3OBJ, Moon reports fewer #**MFC**, #**PCS**, #**RM**, or #**CE** than DebloaterX, indicating that Moon may achieve higher precision in certain scenarios, therefore compensating for precision losses due to the heuristics nature of DebloaterX.

In terms of analysis efficiency, Cut-Shortcut achieves the highest overall speedup of >41.8X and >299.7X in comparison to 2OBJ and 3OBJ, much faster than Zipper (>2.8X and >4.0X), Conch (>3.5X and >4.8X), and DebloaterX (>17.6X and >150.0X). It is important to emphasize that Moon achieves an impressive overall speedup of >37.2X and >382.0X, comparable with Cut-Shortcut's speedup for 2OBJ and yields even higher speedup for 3OBJ, as depicted in Figure 10a and 10b. Specifically, Moon demonstrates a more efficient analysis than Cut-Shortcut, achieving speedups of 18.6% to 294.2% across 23 programs for 2OBJ, and 2.2% to 113.7% across 21 programs for 3OBJ. Furthermore, the programs where Moon and Cut-Shortcut demonstrate peak efficiency quite differ. It is reasonable, since Cut-Shortcut mainly relies on context insensitivity and simulating context sensitivity by proposing three patterns that address partible, but not all precision loss scenarios, resulting in a high efficiency. In contrast, our Moon is grounded in a theoretically proven framework that identifies a superset of objects that can improve precision, ensuring almost



(a) Speedup of Cut-Shortcut, Z-2obj, C-2obj, D-2obj, and M-2obj.



(b) Speedup of Cut-Shortcut, Z-3obj, C-3obj, D-3obj, and M-3obj.

Fig. 10. The speedups of Cut-Shortcut and Zipper-, Conch-, DebloaterX-, and Moon-guided pointer analysis on 2OBJ (a) and 3OBJ (b) over conventional analysis.

full preservation of precision regardless of whether they are efficiency-critical or not. Therefore, Moon achieves significantly lower precision loss than Cut-Shortcut across all 30 programs. Additionally, compared to DebloaterX, which ranks second in efficiency, Moon achieves an overall speedup improvement of 111.3% and 154.6% for 2OBJ and 3OBJ while maintaining comparable precision. Across all 30 programs, the speedup improvement achieved by Moon over DebloaterX is substantial and consistently maintained, ranging from 21.9% to 216.9% for 2OBJ. A potential explanation for Moon's considerably higher efficiency may be observed in Figure 8, which illustrates that DebloaterX marks 7.9% of objects as requiring context sensitivity—approximately five times the percentage identified by Moon for 2OBJ. Furthermore, M-3obj shows an even greater speedup over D-3obj, which encounters scalability issues on three programs, while Moon completes each analysis within approximately one hour.

In summary, Moon exhibits the exceptional efficiency of Cut-Shortcut alongside the high precision of DebloaterX, while outperforming both approaches in either precision or efficiency.

6.3 RQ3: Computational Overhead of Moon Compared to Other Selective Heuristics

Selective pointer analysis techniques, such as Zipper, Conch, DebloaterX, and Moon, typically follow a multi-stage pipeline, which can be divided into three main stages: **(1)** a context-insensitive analysis, **(2)** a selection process guided by heuristics, and **(3)** the final selective context-sensitive analysis. Notably, Cut-Shortcut performs the selection implicitly during analysis (i.e., selection on-the-fly), and thus consists solely of the final stage. Therefore, for selective approaches, the first two stages (referred to as the **pre-analysis** stage) constitute additional computational overhead when compared to conventional context-sensitive pointer analysis.

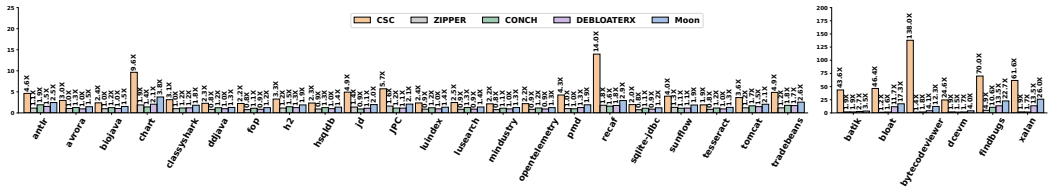
6.3.1 Comparison of Overhead Introduced by Pre-Analysis. Table 4 presents the overhead associated with Moon's pre-analysis, whereas the pre-analysis time of Zipper, Conch, and DebloaterX are also included for comparison. While Moon dynamically adjusts to 2OBJ and 3OBJ sensitivities, the difference in execution time between the two configurations is negligible, so we present the average overhead in Table 4 for clarity.

Table 4. Pre-analysis time comparison of Zipper, Conch, DebloaterX, and Moon, whereas CI is presented as a baseline.

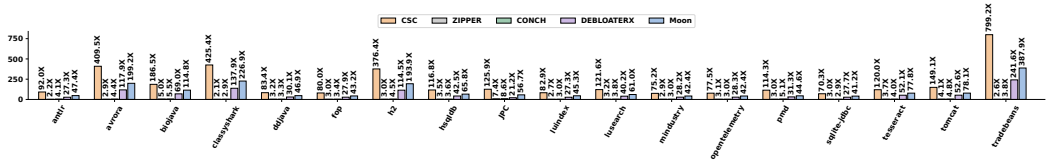
Program	CI	Zipper	Conch	DebloaterX	**Moon**	Program	CI	Zipper	Conch	DebloaterX	**Moon**	Program	CI	Zipper	Conch	DebloaterX	**Moon**
antlr	4.8	10.7	**7.6**	11.8	8.4	ddjava	4.1	8.0	**5.9**	8.4	6.6	mindustry	3.8	7.7	**5.4**	7.7	6.0
avrora	5.5	11.9	**9.1**	15.7	10.2	eclipse	18.8	208.7	226.4	76.5	**46.8**	opentelemetry	3.9	7.3	**5.1**	7.6	6.1
batik	13.3	57.2	55.1	51.4	**30.7**	findbugs	9.7	31.9	32.2	45.3	**27.0**	pmd	6.5	17.4	15.8	19.1	**12.6**
biojava	4.5	9.1	**6.3**	9.6	7.2	fop	4.0	8.6	**6.1**	9.2	6.4	recaf	20.4	95.2	93.4	106.7	**58.6**
bloat	5.5	11.7	11.8	15.1	**11.0**	h2	4.6	8.8	**6.6**	9.8	7.2	sqlite-jdbc	3.8	7.3	**5.1**	7.5	5.9
bytecodeviewer	9.4	**24.1**	30.8	39.1	30.0	hsqldb	4.4	8.6	**6.0**	8.7	6.9	sunflow	8.2	18.0	21.6	23.5	**15.2**
chart	9.2	37.0	27.8	30.3	**18.7**	jd	10.0	26.1	42.2	33.0	**20.4**	tesseract	4.6	9.3	**6.9**	9.6	7.5
checkstyle	8.2	43.2	17.8	25.8	**17.3**	JPC	10.3	21.5	26.7	30.2	**18.3**	tomcat	4.0	8.6	**5.9**	8.7	6.7
classyshark	5.3	13.6	11.6	13.7	**9.8**	luindex	3.9	8.0	**5.5**	8.4	6.4	tradebeans	4.7	11.2	8.2	11.1	**7.9**
dcevm	13.5	52.3	58.6	49.2	**29.9**	lusearch	4.2	8.9	**5.8**	10.0	6.8	xalan	5.2	16.8	**9.4**	13.6	9.5

Moon achieves an average pre-analysis time of just 15.4s, representing only 57.1%, 59.5%, and 64.5% of the pre-analysis time required by Zipper, Conch, and DebloaterX, respectively, demonstrating its superior efficiency. Furthermore, on 14 out of 30 benchmarks, Moon 's pre-analysis is the fastest among all approaches. On the remaining benchmarks, it still performs comparably to the most efficient method. While Moon incurs relatively higher pre-analysis costs on certain large programs such as *eclipse* and *batik*, these figures remain relatively small compared to those required by the corresponding selective pointer analyses—245.5s and 201.7s, respectively.

6.3.2 Comparison of Speedups of Entire Analysis. Figure 11a and 11b illustrate the overall speedups of the entire analysis (including all three stages) of Cut-Shortcut, Zipper, Conch, DebloaterX and Moon, with the detailed statistics provided in our supplementary material. These results show that Moon achieves average speedups of >12.0X and >109.1X over conventional object-sensitive analysis for 2OBJ and 3OBJ, respectively, still significantly faster than DebloaterX (>6.9X and >61.1X), Conch (>1.8X and >4.5X) and Zipper (>1.4X and >3.7X), only slower than Cut-Shortcut (>41.8X and >299.7X). The performance gap between Moon and Cut-Shortcut stems from fundamental differences in methodology and goals. Cut-Shortcut begins from context insensitivity and incrementally refines PFG edges to simulate context sensitivity, prioritizing efficiency. In contrast, Moon begins from full context sensitivity and aims to eliminate redundant analysis overhead, prioritizing precision. This distinction is reflected in the experiments: while Cut-Shortcut sacrifices substantial precision (62.8% and 66.9% losses), Moon retains nearly all of it (only 0.1% and 0.2% loss).



(a) Speedup of Cut-Shortcut, Zipper, Conch, DebloaterX, and Moon under 2OBJ.



(b) Speedup of Cut-Shortcut, Zipper, Conch, DebloaterX, and Moon under 3OBJ.

Fig. 11. Speedups of the entire analysis (including all three stages) of Cut-Shortcut, Zipper, Conch, DebloaterX, and Moon under 2OBJ (a) and 3OBJ (b), over conventional context-sensitive analysis.

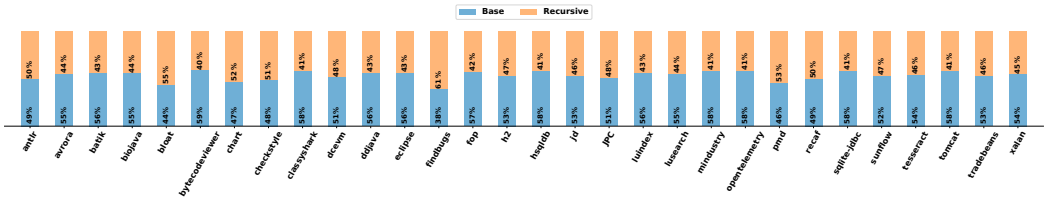
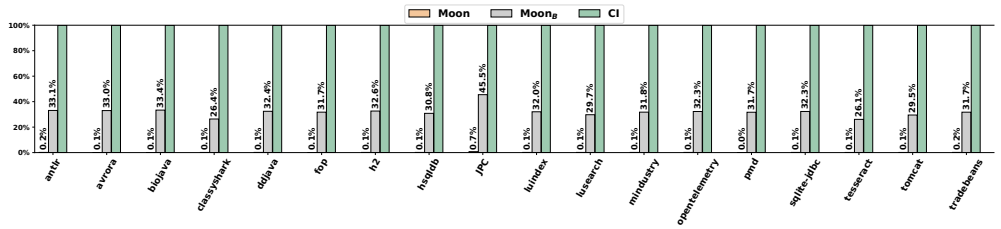
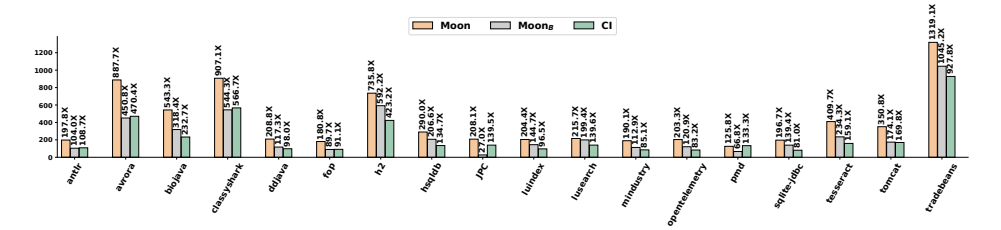


Fig. 12. Distributions of two types of *Applied Precision-Relevant* objects for 3OBJ.



(a) Percentages of precision losses of Moon and its variant.



(b) Speedups of Moon and its variant.

Fig. 13. Overall precision losses and speedups of Moon and its variant for 3OBJ.

6.4 RQ4: Contribution of Two Types of Applied Precision-Relevant Objects in Moon

Here we present only 3OBJ results for the ablation experiment in addressing RQ4, as 3OBJ results provide a more comprehensive view of the impact of each object type. Figure 12 illustrates the distribution of *Applied Precision-Relevant* objects identified by Moon for 3OBJ. Across all 30 programs, *Base Precision-Relevant* objects dominate usually, occupying 39.0%-59.2% of all objects, with an average of 53.6%. Figure 13 further demonstrates the precision loss and efficiency of Moon and its two variants on 18 programs that 3OBJ successfully finishes. Moon$_B$ exhibits a precision losses of 32.0% and a speedup of 260.4X. These figures indicate that *Base* objects preserves only a number of, around 68.0% precision, though covering this object type do slow down the analysis speed (CI achieves a speedup of 360.9X on average for comparison).

7 Discussion

Generalization of the Proposed Theory and Approximations. Our theory formalizes the conditions under which heap objects contribute to precision based on reachability in PFGs, making it agnostic to specific context-sensitivity strategies. However, our approximations rely on Theorem 4.1, which holds only under object-sensitivity, thus making the current approximations specific to object-sensitivity. Meanwhile, although our implementation targets Java, the core insight is grounded in the object encapsulation principle common to many object-oriented languages. As such, our approach is applicable to other languages that follow the object-oriented paradigm.

Directions for Further Work. Our framework still applies context sensitivity in an all-or-nothing manner, i.e., objects are either fully context-sensitive or entirely context-insensitive. A promising extension is to allow partial application of context sensitivity. Formalizing this partial application and integrating it into our reachability-based theory would be an important next step toward more flexible and efficient pointer analysis.

8 Related Work

We discuss related work that aims to improve conventional context sensitivity in this Section.

Object Selection. For certain objects, context sensitivity does not necessarily improve precision but introduce unnecessary overhead. Moon applies context sensitivity only to specific objects where it is beneficial. Before Moon, Conch [8] was the first to propose this idea, employing pattern identification through an IFDS-based algorithm. Subsequently, DebloaterX [7] extended this method by incorporating a container-usage-pattern-based approach. However, both approaches depend on empirical inspection of patterns. In contrast, our work provides a deeper understanding of the underlying mechanisms, leading to the development of a simple yet effective graph traversal algorithm. This algorithm facilitates an extremely fast and precise selective context-sensitive pointer analysis. Mahjong [38], also manipulates objects by merging type-consistent objects through the combination of equivalent automata. This approach targets precision improvements in type-dependent information.

Method Selection. Like objects, certain methods in programs add overhead without improving precision when treated with context sensitivity. This idea originated from Introspective Analysis [31], which used parameterized heuristics to guide method selection. Various strategies have since been developed, including machine-learning approaches [11, 13, 15], memory abstraction based on capacity [20], program patterns [19, 21], and more comprehensive selection schemes like [36].

Context Element Selection. Conventional context sensitivity uses k-limiting consecutive context elements to represent method contexts. Bean [37] identified redundant context elements and eliminate them using an object allocation graph. Similarly, context tunneling [12] employs machine learning and [14] builds on this by transforming *Object-Sensitivity* to *CallSite-Sensitivity*, demonstrating that the latter can simulate the former but not vice versa. Beyond selecting specific context elements, other approaches hybridize different kinds of context elements. [18] combines *CallSite-Sensitivity* and *Object-Sensitivity* elements, leading to more effective results. [40] combines *Object-Sensitivity* contexts with level-summarized relevant value contexts, as proposed in [39].

Other Related Work. Cut-Shortcut [25] is a recent approach that modifies the pointer flow graph during the on-the-fly propagation of a context-insensitive analysis, similar algorithm to modify the input graph to express various analysis problems [10, 23, 32, 33]. In addition to context sensitivity, [4] partially perform strong updates to objects via a flow-sensitive approach, accomplishing by expressing points-to relations via access paths [5, 16]. A saturation-based points-to analysis is proposed in [45], which achieves scalability while retaining much of the precision of context-insensitive points-to analysis.

9 Conclusions

In this paper, we presented Moon, a principled approach to identifying *Precision-Relevant* objects, with the goal of maintaining precision while decreasing the overhead introduced by context sensitivity. The conceptual contribution of this work includes a formal theory that explains how precision is preserved using object contexts, supported by a rigorous proof. The practical contribution lies in the effective implementation for identifying these objects. We have conducted experiments on a wide range of benchmarks, demonstrating its effectiveness on real-world Java programs.

Data-Availability Statement

The source code, artifacts, reproducible data reported in Section 6, and detailed formal proofs of all theorems and lemmas are publicly available in our artifact repository [46].

Acknowledgments

We thank the anonymous reviewers for their constructive feedback. This research was supported by the National Natural Science Foundation of China under Grant Nos. 62372227 and 62402213.

References

[1] Stephen M Blackburn, Zixian Cai, Rui Chen, Xi Yang, John Zhang, and John Zigman. 2025. Rethinking Java Performance Analysis. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*. ACM. doi:10.1145/3669940.3707217

[2] S. M. Blackburn, R. Garner, C. Hoffman, A. M. Khan, K. S. McKinley, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, M. Hirzel, A. Hosking, M. Jump, H. Lee, J. E. B. Moss, A. Phansalkar, D. Stefanović, T. VanDrunen, D. von Dincklage, and B. Wiedermann. 2006. The DaCapo Benchmarks: Java Benchmarking Development and Analysis. In *OOPSLA '06: Proceedings of the 21st annual ACM SIGPLAN conference on Object-Oriented Programing, Systems, Languages, and Applications* (Portland, OR, USA). ACM Press, New York, NY, USA, 169–190. doi:10.1145/1167473.1167488

[3] Yuandao Cai, Chengfeng Ye, Qingkai Shi, and Charles Zhang. 2022. Peahen: fast and precise static deadlock detection via context reduction. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 784–796. doi:10.1145/3540250.3549110

[4] Arnab De and Deepak D'Souza. 2012. Scalable Flow-Sensitive Pointer Analysis for Java with Strong Updates. In *ECOOP 2012 – Object-Oriented Programming*, James Noble (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 665–687.

[5] Stephen J. Fink, Eran Yahav, Nurit Dor, G. Ramalingam, and Emmanuel Geay. 2008. Effective typestate verification in the presence of aliasing. *ACM Trans. Softw. Eng. Methodol.* 17, 2, Article 9 (May 2008), 34 pages. doi:10.1145/1348250.1348255

[6] Behnaz Hassanshahi, Raghavendra Kagalavadi Ramesh, Padmanabhan Krishnan, Bernhard Scholz, and Yi Lu. 2017. An efficient tunable selective points-to analysis for large codebases. In *Proceedings of the 6th ACM SIGPLAN International Workshop on State Of the Art in Program Analysis* (Barcelona, Spain) *(SOAP 2017)*. Association for Computing Machinery, New York, NY, USA, 13–18. doi:10.1145/3088515.3088519

[7] Dongjie He, Yujiang Gui, Wei Li, Yonggang Tao, Changwei Zou, Yulei Sui, and Jingling Xue. 2023. A Container-Usage-Pattern-Based Context Debloating Approach for Object-Sensitive Pointer Analysis. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 256 (oct 2023), 30 pages. doi:10.1145/3622832

[8] Dongjie He, Jingbo Lu, and Jingling Xue. 2021. Context Debloating for Object-Sensitive Pointer Analysis. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 79–91. doi:10.1109/ASE51524.2021.9678880

[9] Dongjie He, Jingbo Lu, and Jingling Xue. 2022. Qilin: A New Framework For Supporting Fine-Grained Context-Sensitivity in Java Pointer Analysis. In *36th European Conference on Object-Oriented Programming (ECOOP 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 222)*, Karim Ali and Jan Vitek (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 30:1–30:29. doi:10.4230/LIPIcs.ECOOP.2022.30

[10] Wei Huang, Yao Dong, Ana Milanova, and Julian Dolby. 2015. Scalable and precise taint analysis for Android. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis* (Baltimore, MD, USA) *(ISSTA 2015)*. Association for Computing Machinery, New York, NY, USA, 106–117. doi:10.1145/2771783.2771803

[11] Minseok Jeon, Sehun Jeong, Sungdeok Cha, and Hakjoo Oh. 2019. A Machine-Learning Algorithm with Disjunctive Model for Data-Driven Program Analysis. 41, 2, Article 13 (June 2019), 41 pages. doi:10.1145/3293607

[12] Minseok Jeon, Sehun Jeong, and Hakjoo Oh. 2018. Precise and scalable points-to analysis via data-driven context tunneling. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 140 (Oct. 2018), 29 pages. doi:10.1145/3276510

[13] Minseok Jeon, Myungho Lee, and Hakjoo Oh. 2020. Learning graph-based heuristics for pointer analysis without handcrafting application-specific features. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 179 (Nov. 2020), 30 pages. doi:10.1145/3428247

[14] Minseok Jeon and Hakjoo Oh. 2022. Return of CFA: call-site sensitivity can be superior to object sensitivity even for object-oriented programs. *Proc. ACM Program. Lang.* 6, POPL, Article 58 (Jan. 2022), 29 pages. doi:10.1145/3498720

[15] Sehun Jeong, Minseok Jeon, Sungdeok Cha, and Hakjoo Oh. 2017. Data-driven context-sensitivity for points-to analysis. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 100 (oct 2017), 28 pages. doi:10.1145/3133924

[16] Vini Kanvar and Uday P. Khedker. 2016. Heap Abstractions for Static Analysis. *ACM Comput. Surv.* 49, 2, Article 29 (June 2016), 47 pages. doi:10.1145/2931098

[17] George Kastrinis and Yannis Smaragdakis. 2013. Hybrid context-sensitivity for points-to analysis. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Seattle, Washington, USA) *(PLDI '13)*. Association for Computing Machinery, New York, NY, USA, 423–434. doi:10.1145/2491956.2462191

[18] George Kastrinis and Yannis Smaragdakis. 2013. Hybrid context-sensitivity for points-to analysis. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Seattle, Washington, USA) *(PLDI '13)*. Association for Computing Machinery, New York, NY, USA, 423–434. doi:10.1145/2491956.2462191

[19] Yue Li, Tian Tan, Anders Møller, and Yannis Smaragdakis. 2018. Precision-guided context sensitivity for pointer analysis. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 141 (Oct. 2018), 29 pages. doi:10.1145/3276511

[20] Yue Li, Tian Tan, Anders Møller, and Yannis Smaragdakis. 2018. Scalability-first pointer analysis with self-tuning context-sensitivity *(ESEC/FSE 2018)*. Association for Computing Machinery, New York, NY, USA, 129–140. doi:10.1145/3236024.3236041

[21] Yue Li, Tian Tan, Anders Møller, and Yannis Smaragdakis. 2020. A Principled Approach to Selective Context Sensitivity for Pointer Analysis. *ACM Trans. Program. Lang. Syst.* 42, 2, Article 10 (May 2020), 40 pages. doi:10.1145/3381915

[22] Yue Li, Tian Tan, Yifei Zhang, and Jingling Xue. 2016. Program Tailoring: Slicing by Sequential Criteria. In *30th European Conference on Object-Oriented Programming (ECOOP 2016) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 56)*, Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 15:1–15:27. doi:10.4230/LIPIcs.ECOOP.2016.15

[23] Yuanbo Li, Qirun Zhang, and Thomas Reps. 2022. Fast Graph Simplification for Interleaved-Dyck Reachability. *ACM Trans. Program. Lang. Syst.* 44, 2, Article 11 (May 2022), 28 pages. doi:10.1145/3492428

[24] Jingbo Lu and Jingling Xue. 2019. Precision-preserving yet fast object-sensitive pointer analysis with partial context sensitivity. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 148 (Oct. 2019), 29 pages. doi:10.1145/3360574

[25] Wenjie Ma, Shengyuan Yang, Tian Tan, Xiaoxing Ma, Chang Xu, and Yue Li. 2023. Context Sensitivity without Contexts: A Cut-Shortcut Approach to Fast and Precise Pointer Analysis. *Proc. ACM Program. Lang.* 7, PLDI, Article 128 (jun 2023), 26 pages. doi:10.1145/3591242

[26] Ana Milanova, Atanas Rountev, and Barbara G. Ryder. 2002. Parameterized object sensitivity for points-to and side-effect analyses for Java. *SIGSOFT Softw. Eng. Notes* 27, 4 (jul 2002), 1–11. doi:10.1145/566171.566174

[27] Ana Milanova, Atanas Rountev, and Barbara G. Ryder. 2005. Parameterized object sensitivity for points-to analysis for Java. *ACM Trans. Softw. Eng. Methodol.* 14, 1 (jan 2005), 1–41. doi:10.1145/1044834.1044835

[28] Mayur Naik, Alex Aiken, and John Whaley. 2006. Effective static race detection for Java. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Ottawa, Ontario, Canada) *(PLDI '06)*. Association for Computing Machinery, New York, NY, USA, 308–319. doi:10.1145/1133981.1134018

[29] Yannis Smaragdakis and George Balatsouras. 2015. Pointer Analysis. *Foundations and Trends® in Programming Languages* 2, 1 (2015), 1–69. doi:10.1561/2500000014

[30] Yannis Smaragdakis, Martin Bravenboer, and Ondrej Lhoták. 2011. Pick your contexts well: understanding object-sensitivity. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Austin, Texas, USA) *(POPL '11)*. Association for Computing Machinery, New York, NY, USA, 17–30. doi:10.1145/1926385.1926390

[31] Yannis Smaragdakis, George Kastrinis, and George Balatsouras. 2014. Introspective analysis: context-sensitivity, across the board. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Edinburgh, United Kingdom) *(PLDI '14)*. Association for Computing Machinery, New York, NY, USA, 485–495. doi:10.1145/2594291.2594320

[32] Johannes Späth, Lisa Nguyen Quang Do, Karim Ali, and Eric Bodden. 2016. Boomerang: Demand-Driven Flow- and Context-Sensitive Pointer Analysis for Java. In *30th European Conference on Object-Oriented Programming (ECOOP 2016) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 56)*, Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 22:1–22:26. doi:10.4230/LIPIcs.ECOOP.2016.22

[33] Manu Sridharan and Rastislav Bodík. 2006. Refinement-based context-sensitive points-to analysis for Java. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Ottawa, Ontario, Canada) *(PLDI '06)*. Association for Computing Machinery, New York, NY, USA, 387–400. doi:10.1145/1133981.1134027

[34] Manu Sridharan, Satish Chandra, Julian Dolby, Stephen J. Fink, and Eran Yahav. 2013. *Alias Analysis for Object-Oriented Programs*. Springer Berlin Heidelberg, Berlin, Heidelberg, 196–232. doi:10.1007/978-3-642-36946-9_8

[35] Manu Sridharan, Stephen J. Fink, and Rastislav Bodik. 2007. Thin slicing. *SIGPLAN Not.* 42, 6 (June 2007), 112–122. doi:10.1145/1273442.1250748

[36] Tian Tan, Yue Li, Xiaoxing Ma, Chang Xu, and Yannis Smaragdakis. 2021. Making pointer analysis more precise by unleashing the power of selective context sensitivity. 5, OOPSLA, Article 147 (Oct. 2021), 27 pages. doi:10.1145/3485524

[37] Tian Tan, Yue Li, and Jingling Xue. 2016. Making k-Object-Sensitive Pointer Analysis More Precise with Still k-Limiting. In *Static Analysis*, Xavier Rival (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 489–510.

[38] Tian Tan, Yue Li, and Jingling Xue. 2017. Efficient and precise points-to analysis: modeling the heap by merging equivalent automata. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Barcelona, Spain) *(PLDI 2017)*. Association for Computing Machinery, New York, NY, USA, 278–291. doi:10.1145/3062341.3062360

[39] Manas Thakur and V. Krishna Nandivada. 2019. Compare less, defer more: scaling value-contexts based whole-program heap analyses. In *Proceedings of the 28th International Conference on Compiler Construction* (Washington, DC, USA) *(CC 2019)*. Association for Computing Machinery, New York, NY, USA, 135–146. doi:10.1145/3302516.3307359

[40] Manas Thakur and V. Krishna Nandivada. 2020. Mix your contexts well: opportunities unleashed by recent advances in scaling context-sensitivity. In *Proceedings of the 29th International Conference on Compiler Construction* (San Diego, CA, USA) *(CC 2020)*. Association for Computing Machinery, New York, NY, USA, 27–38. doi:10.1145/3377555.3377902

[41] Rei Thiessen and Ondřej Lhoták. 2017. Context transformations for pointer analysis. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Barcelona, Spain) *(PLDI 2017)*. Association for Computing Machinery, New York, NY, USA, 263–277. doi:10.1145/3062341.3062359

[42] Paolo Tonella. 2005. Reverse engineering of object oriented code. In *Proceedings of the 27th International Conference on Software Engineering* (St. Louis, MO, USA) *(ICSE '05)*. Association for Computing Machinery, New York, NY, USA, 724–725. doi:10.1145/1062455.1062637

[43] Paolo Tonella and Alessandra Potrich. 2005. *Reverse Engineering of Object Oriented Code*. Springer. doi:10.1007/b102522

[44] WALA. 2006. Watson Libraries for Analysis. http://wala.sf.net

[45] Christian Wimmer, Codrut Stancu, David Kozak, and Thomas Würthinger. 2024. Scaling Type-Based Points-to Analysis with Saturation. *Proc. ACM Program. Lang.* 8, PLDI, Article 187 (June 2024), 24 pages. doi:10.1145/3656417

[46] Chaoyue Zhang, Longlong Lu, Yifei Lu, Minxue Pan, and Xuandong Li. 2025. *Towards a Theoretically-Backed and Practical Framework for Selective Object-Sensitive Pointer Analysis (Artifact and Supplemental Materials)*. doi:10.5281/zenodo.16890176

Received 2025-03-26; accepted 2025-08-12